影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 13 章 特徵擷取

本章學習目標

- 說明「特徵」在電腦視覺中的意義，理解好的特徵應具備重複性、獨特性與不變性。
- 推導 Harris 與 Shi-Tomasi 角點偵測，說明角點為何是良好的追蹤與匹配基準。
- 闡述 SIFT 的尺度不變原理與 ORB 的工程取捨，說明兩者的授權與效能差異。
- 運用 BF 與 FLANN 進行特徵匹配，並以比值測試與 RANSAC 去除錯誤匹配。
- 完成影像拼接、模板比對與物件定位，並實作車牌偵測的特徵前處理。
- 運用 HOG、LBP 與形狀特徵描述整塊區域與物件外形，並說明三類手工特徵的分工與選用時機。
- 理解手工特徵與深度學習特徵的關係，說明本章在傳統與現代視覺之間的橋梁地位。

叫獸開講|SIFT 的二十年專利長跑：一個演算法的解禁之日

2020 年 3 月，全球電腦視覺社群悄悄迎來一個值得慶祝的日子。開源函式庫 OpenCV 的開發者們，終於可以名正言順地把一個名叫 SIFT 的演算法，從那個帶著警示意味的「非自由」(non-free)模組，搬進人人可自由使用的主庫。這一天，許多工程師等了將近二十年。

故事要從 1999 年說起。加拿大英屬哥倫比亞大學的 David Lowe 教授，發表了尺度不變特徵轉換(Scale-Invariant Feature Transform，簡稱 SIFT)。這是一個劃時代的成就：在 SIFT 之前，電腦視覺一直被一個根本難題困擾——當同一個物

體以不同的大小、不同的角度、不同的光線出現時，電腦很難認出「這是同一個東西」。SIFT 優雅地解決了這個問題，它能在影像中找出一些獨特的「關鍵點」，而這些關鍵點對縮放、旋轉都保持不變，對光線變化與雜訊也相當強健——就像替影像中的每個特徵蓋上一枚無論如何變形都認得出來的「指紋」。SIFT 迅速成為物件辨識、影像拼接、3D 重建的基石，是深度學習興起前最重要的視覺演算法之一。

但 SIFT 有一道緊箍咒：它被英屬哥倫比亞大學申請了專利(美國專利 6,711,293,1999 年 3 月提出臨時申請，2004 年正式授予)。這意味著在商業產品中使用 SIFT 需要授權。因此，儘管 OpenCV 收錄了 SIFT 的實作，卻只能把它放進一個特別的「非自由」模組，並附上明確警示：使用前請自行確認專利與授權問題。全球工程師只能一邊讚嘆它的威力，一邊小心翼翼地繞道而行。這道緊箍咒也催生了替代品：2011 年，OpenCV 團隊自己提出了 ORB(Oriented FAST and Rotated BRIEF)，一個完全免費、速度更快、專為「沒有專利負擔」而設計的方案——工程界對專利限制的回應，往往就是造一個自由的輪子。

專利的保護期是二十年。2020 年 3 月，SIFT 的專利終於到期。消息傳開，社群一片歡騰，OpenCV 隨即在 4.4.0(以及 3.4.11)版本中，把 SIFT 正式移入自由的主庫。一個沉睡了二十年的經典，終於解除封印，回到每一位工程師都能自由取用的工具箱中。

這個故事給了我們兩個超越技術的啟示。其一，技術選型不只看效能，還要看授權：在 SIFT 被專利綁住的年代，免費的 ORB 儘管精度稍遜，卻因為「可以自由商用」而被廣泛採用——工程決策從來不是在真空中做的。其二，好的基礎研究會持續發光：即使深度學習已經興起，SIFT 所奠定的「尺度不變、局部特徵」思想，依然深深影響著今天的視覺演算法。這一章，我們就來認識這些為影像蓋「指紋」的經典方法。

思考題：在 SIFT 被專利保護的二十年間，ORB 這個「免費替代品」蓬勃發展。

> 你能想到其他領域中，因為某個技術被專利或授權限制，而催生出自由替代方案的例子嗎?(提示：作業系統、資料庫、影音編碼)

13.1 什麼是好的特徵

13.1.1 從像素到特徵：再一次的抽象躍升

回顧全書的旅程：第 1 到 8 章我們處理「像素」，第 9、10 章我們把像素聚成「區域與物件」，而本章要再往上一階——從影像中萃取出精簡而有代表性的「特徵」(Feature)。所謂特徵，是影像中一小塊「值得注意、容易辨認、可重複找到」的局部結構。與其比對成千上萬個像素，不如只比對數十個關鍵特徵——這讓影像的比對、辨識、對齊變得高效而強健。

這個抽象躍升的價值，可以用一個生活比喻理解：要向朋友描述一個人，你不會逐一報告他臉上每個毛孔的位置與顏色(像素層級)，而是說「他有濃眉、鷹勾鼻、左臉有顆痣」(特徵層級)——用少數幾個獨特的標記，就足以精準辨認。電腦視覺的特徵擷取，做的正是同一件事：為影像找出它最具辨識度的「標記」。

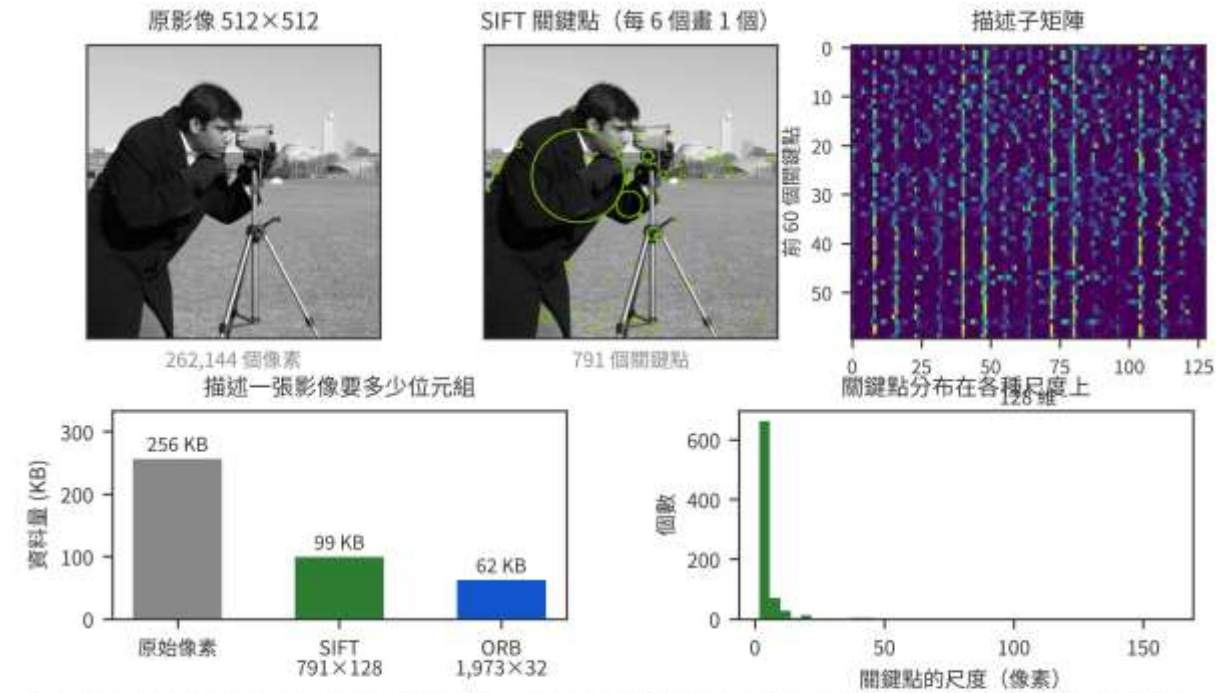
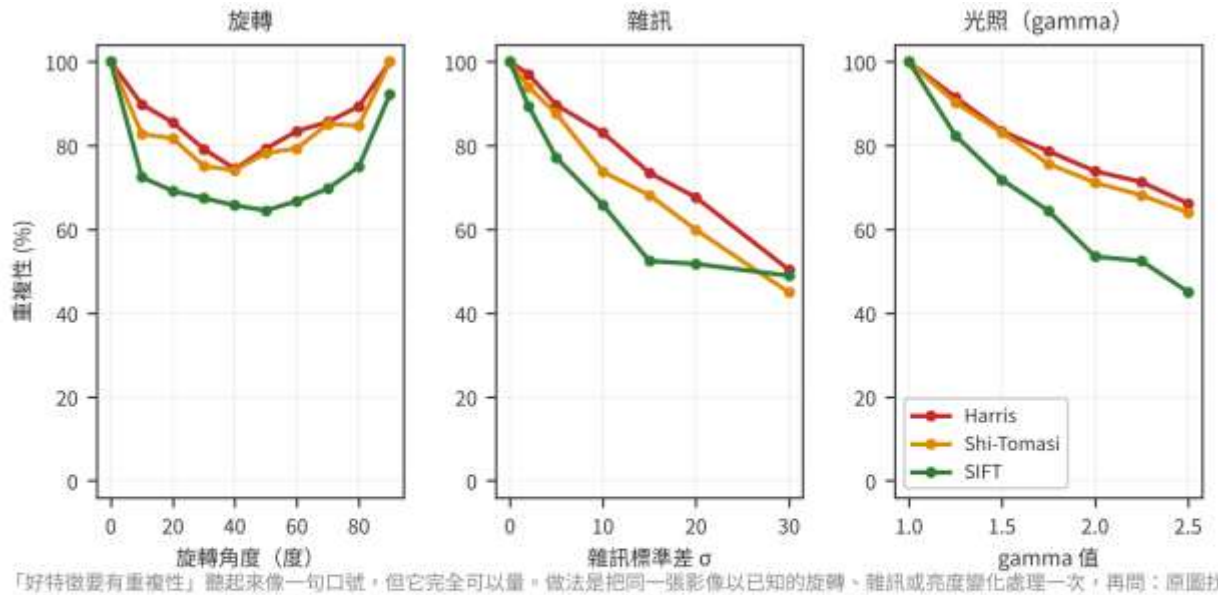


圖 13-1　從像素到特徵：資料量少了，能做的事卻多了

13.1.2 好特徵的三個條件

並非影像中任何一點都適合當特徵。一個理想的特徵應具備三個性質：

1. 重複性(Repeatability)：同一個場景在不同條件下(不同視角、光線、雜訊)拍攝，同一個特徵都應該能被再次偵測到。若一個特徵這次找得到、下次就消失，它就無法用於比對。
2. 獨特性(Distinctiveness)：特徵應該與眾不同、容易與其他特徵區分。一片平坦的牆面、一段筆直的邊緣都不夠獨特(牆上每一點都長得一樣，邊緣上的點沿邊緣滑動也分不出來)，而角點、紋理豐富的斑塊則獨一無二。
3. 不變性(Invariance)：特徵的描述應該對各種變化保持穩定——尺度不變(物體放大縮小仍認得)、旋轉不變(物體轉了角度仍認得)、光照不變(明暗改變仍認得)。不變性愈強，特徵愈能在惡劣條件下發揮作用。



「好特徵要有重複性」聽起來像一句口號，但它完全可以量。做法是把同一張影像以已知的旋轉、雜訊或亮度變化處理一次，再問：原圖找到的關鍵點，經過同樣的轉換之後，有多少比例能在新影像中找到 3 像素以內的夥伴?這個比例就是重複性。三個子圖各自只改變一個變因。結果：旋轉 40 度時 Harris 掉到 74%、Shi-Tomasi 74%、SIFT 66%;雜訊 σ=30 時三者分別是 50%、45%、49%;gamma 拉到 2.5 時則是 66%、64%、45%。有兩件事值得特別注意。第一，旋轉 90 度時重複性會回升到接近 100%，那是因為 90 度只是把像素重新排列、沒有重新取樣，不產生插值誤差——評估不變性時要刻意避開這種「剛好對齊」的特例。第二，SIFT 在這三項測試中都不是最好的，這並不矛盾：旋轉與光照的不變性，Harris 這類基於梯度的偵測器本來就具備（結構張量隨影像一起旋轉）。SIFT 真正的本事在尺度，而尺度正是上面三張圖沒有測的那一項——下一節會補上，屆時排序會完全翻轉。

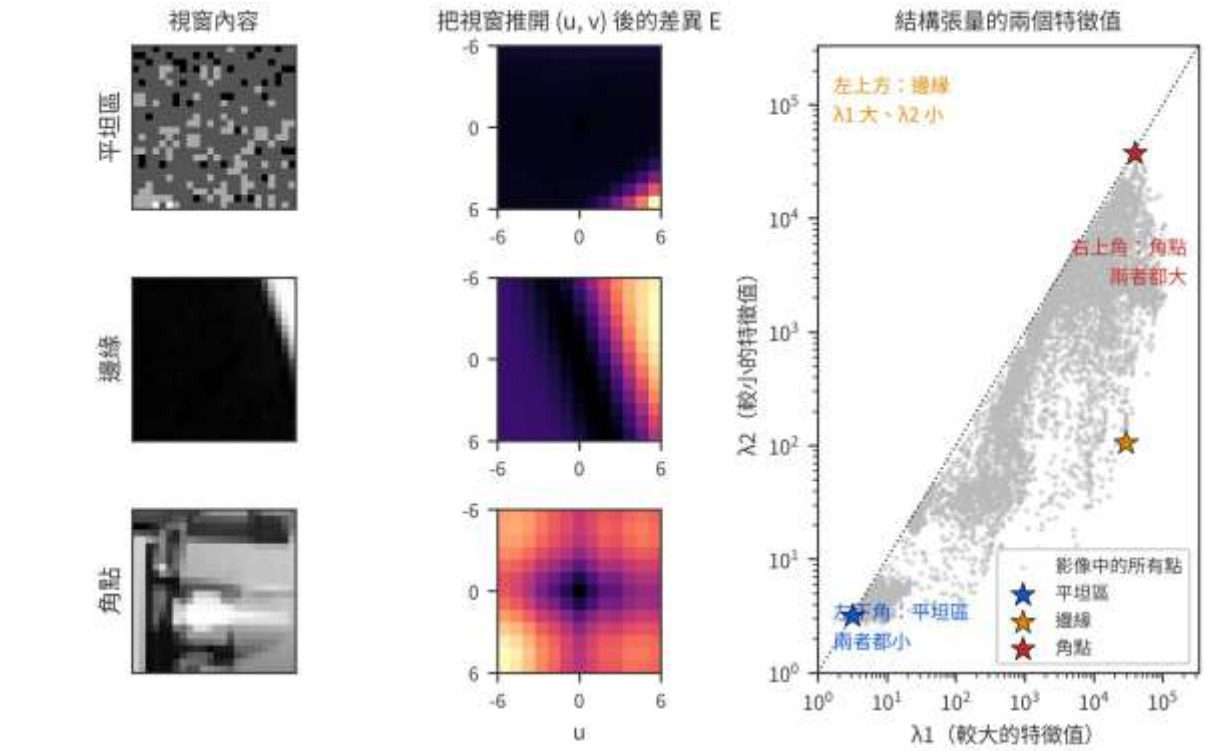
圖 13-2　重複性是可以量的：旋轉、雜訊、光照三項實測

這三個條件也解釋了為什麼「角點」是特徵偵測的寵兒。承接第 12 章的孔徑問題：平坦區域在任何方向移動都看不出變化(無法定位);邊緣只能定出垂直於邊緣的方向(一維模糊);唯有角點在兩個方向都有明顯變化，能被唯一地定位——它同時滿足重複性與獨特性。這就是下一節 Harris 角點偵測的出發點。

13.2 角點偵測：Harris 與 Shi-Tomasi

13.2.1 Harris 角點的直覺

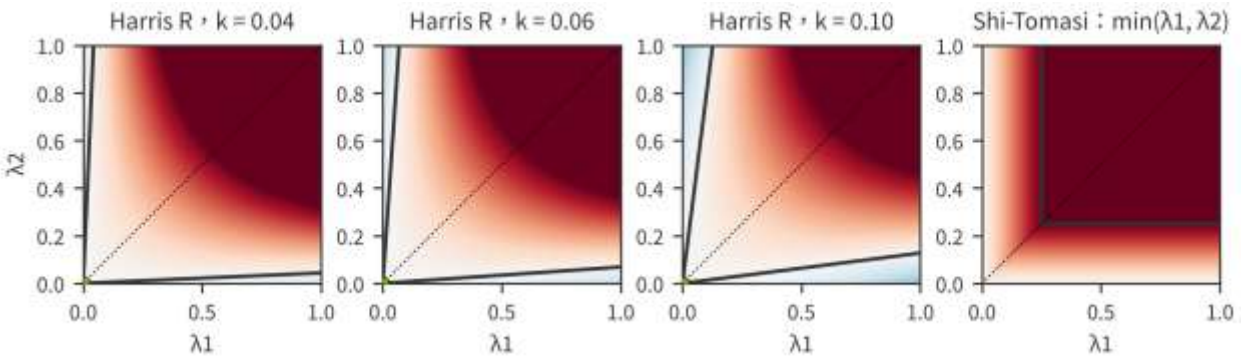
Harris 角點偵測(1988 年由 Chris Harris 與 Mike Stephens 提出)的核心直覺非常優雅：拿一個小視窗在影像上滑動，觀察視窗內的灰階如何變化。有三種情況——若視窗在平坦區域移動，無論往哪個方向，視窗內容都幾乎不變;若視窗沿著邊緣移動(平行於邊緣)，內容不變，但垂直於邊緣移動則劇烈變化;若視窗位於角點，則往任何方向移動，內容都會顯著改變。「往任何方向移動都變化很大」正是角點的數學特徵。



Harris 的直覺是「拿一個小視窗在影像上推來推去，看灰階變化多大」，中間那一欄就把這件事直接畫出來：把視窗往各個方向推 (u, v)，記錄推開之後與原位置的平方差 E。平坦區推到哪裡都幾乎沒變化（整片暗）；邊緣只有垂直於邊的方向會變（一條亮帶）；角點則往任何方向推都劇烈變化（中間一個明顯的坑）。右邊把這件事濃縮成兩個數字——結構張量的兩個特徵值 λ1、λ2，它們正是 E 這個二次曲面在兩個主軸上的陡峭程度。本圖的三個位置不是用眼睛指的，而是由量測挑出來的：λ1 最小者為平坦區（3.2, 3.1，比值 1.0）、λ1 大而比值最高者為邊緣（29120, 105，比值 276）、λ2 最大者為角點（39345, 37386，比值 1.1）。散點圖瞬示絕大多數像素都擠在左下角（平坦）與左上方（邊緣），真正兩者都大的點少之又少——這就是「角點稀少而珍貴」的量化版本。也請留意：這個 2×2 矩陣與第 12 章光流解方程式用的是同一個矩陣，它可逆的條件就是角點的定義。

圖 13-3　平坦、邊緣、角點：自相關曲面與結構張量的兩個特徵值

Harris 把這個直覺量化成一個「結構張量」(從視窗內的梯度統計而來)，其兩個特徵值反映了兩個主要方向上的變化強度：兩者都小是平坦區、一大一小是邊緣、兩者都大則是角點。Harris 巧妙地用一個不需實際計算特徵值的響應函數 R 來判斷，計算高效。這裡也再次呼應第 9 章：梯度(Sobel)是這一切的基礎——角點偵測建立在灰階變化(梯度)的分析之上。



兩個判準都想回答同一個問題——「λ1 與 λ2 是不是都夠大」——但用的方式不同，而這個差別有實際後果。Harris 不去解特徵值（1988 年算特徵值太貴），而是用 $R = \lambda_1\lambda_2 - k(\lambda_1+\lambda_2)^2 = \det(M) - k \cdot \mathrm{trace}(M)^2$ 來近似，因為行列式與跡可以直接從矩陣元素算出來。前三張圖是 R 在 (λ1, λ2) 平面上的樣子，黑線是 R = 0 的分界：k 愈大，被判為角點的區域就愈窄、愈挑剔。要注意 k 有一個上限 0.25——因為在 $\lambda_1 = \lambda_2$ 的對角線上 $R = \lambda^2(1 - 4k)$，一旦 $k > 0.25$，連「完美的角點」R 都會變成負的，整張圖再也找不到角點。最右邊是 Shi-Tomasi：它直接取 $\min(\lambda_1, \lambda_2)$，判準就是「較小的那個夠不夠大」，沒有 k 要調，分界線也單純得多。實測兩者在同一張影像上各取前 200 個角點，有 80% 落在彼此 3 像素以內——大同小異，但 Shi-Tomasi 少一個要調的參數，這就是它成為 goodFeaturesToTrack 預設的原因。

圖 13-4　兩種判準：Harris 的 R 與 Shi-Tomasi 的 min(λ1, λ2)

13.2.2 Shi-Tomasi 的改良

1994 年，Jianbo Shi 與 Carlo Tomasi 在論文《Good Features to Track》中提出改良：與其用 Harris 的響應函數，不如直接取「兩個特徵值中較小者」作為角點強度——只要較小的特徵值夠大，就保證兩個方向都有足夠變化。這個判準在實務上(尤其是用於第 12 章的光流追蹤)表現更穩定，因此 OpenCV 的 goodFeaturesToTrack 函式預設採用 Shi-Tomasi。名稱「適合追蹤的好特徵」也點明了它與第 12 章光流的血緣。

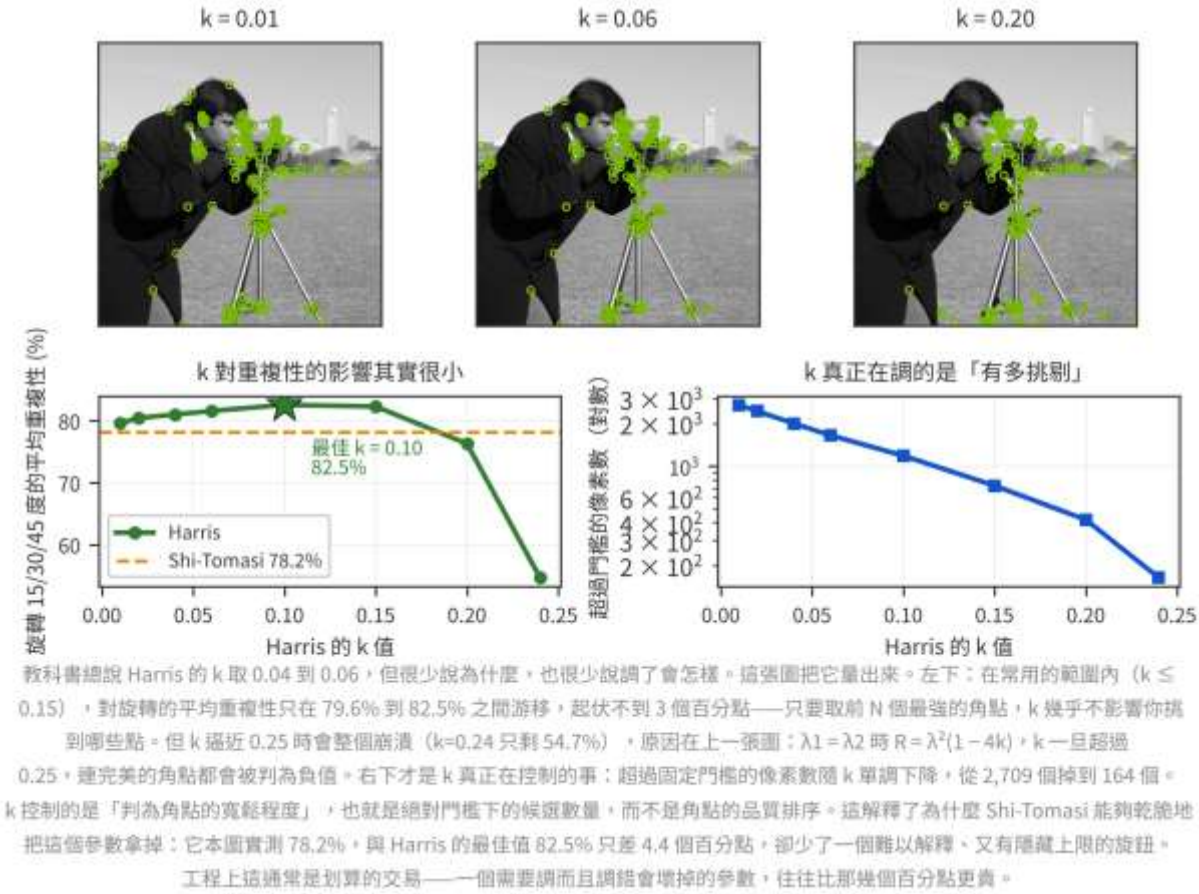


圖 13-5　k 值到底在調什麼：挑剔程度，而不是角點品質

程式 13-1 Harris 與 Shi-Tomasi 角點偵測

```
import cv2, numpy as np

img = cv2.imread("building.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# --- Harris 角點 ---
harris = cv2.cornerHarris(np.float32(gray), blockSize=2,
                          ksize=3, k=0.04)
img_h = img.copy()
img_h[harris > 0.01 * harris.max()] = [0, 0, 255]   # 標紅

# --- Shi-Tomasi(適合追蹤,第 12 章光流的輸入) ---
corners = cv2.goodFeaturesToTrack(gray, maxCorners=100,
            qualityLevel=0.01, minDistance=10)
img_s = img.copy()
for c in np.int0(corners):
    x, y = c.ravel()
    cv2.circle(img_s, (x, y), 4, (0, 255, 0), -1)

cv2.imwrite("harris.png", img_h)
cv2.imwrite("shitomasi.png", img_s)
```

角點偵測的一個侷限是：它不具備尺度不變性。一個在原尺寸下是「尖銳角點」

的結構，尺度一變，同一個固定大小的視窗框住的就是完全不同的內容。這裡要補一個容易被略過的細節：真正垮掉的其實是「描述」而不是「偵測」。實測顯示角點縮小之後多半仍偵測得到、位置也大致還在，但描述子必須知道「要用多大的鄰域來描述這個點」，而固定大小的鄰域在縮放後框住的是不同的結構，於是匹配就失效了(圖 13-6 實測：縮到三分之一時，Harris 配固定區塊描述子的匹配正確率只剩 15%，ORB 57%，而 SIFT 仍有 82%)。要處理「同一物體以不同大小出現」的問題，就需要下一節登場的主角——SIFT。

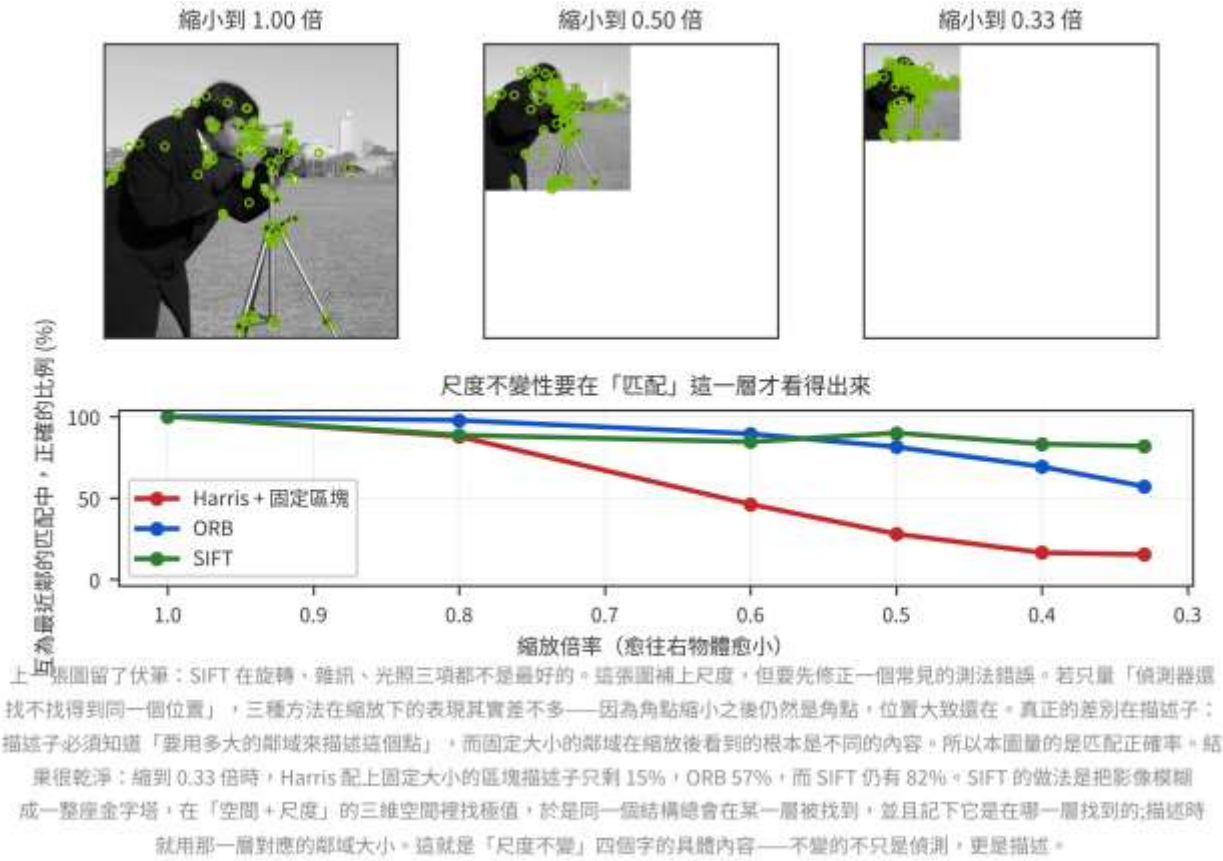


圖 13-6　尺度不變性要在「匹配」這一層才看得出來

13.2.3 推導：Harris 的 R 函數從哪裡來，以及 k 的隱藏上限

13.2.1 節說 Harris 把「視窗往各方向推、看灰階變化多大」量化成一個結構張量，這一節把中間的推導補上，因為它會解釋兩件事：$R = \det(M) - k \cdot \mathrm{trace}(M)^2$ 這個奇怪的式子從哪裡來，以及那個 k 為什麼不能亂調。

先把直覺寫成數學。把視窗往 (u, v) 推之後與原位置的平方差是 $E(u, v) = \Sigma\, w(x, y)\cdot[I(x+u, y+v) - I(x, y)]^2$。對位移很小的情況做一階泰勒展開，$I(x+u, y+v) - I(x, y) \approx I_x\cdot u + I_y\cdot v$，於是 $E(u, v) \approx [u\ v]\cdot M\cdot[u\ v]^T$，其中 M 是那個 2×2 的結構張量，由 I_x^2、I_xI_y、I_y^2 在視窗內的加權和組成。換句話說，E 是一個二次曲面——圖 13-3 中間那一欄畫的就是它。

M 是對稱半正定矩陣，所以可以對角化，兩個特徵值 λ1、λ2 正是這個二次曲面在兩個主軸方向上的陡峭程度。於是判斷規則很清楚：兩者都小是平坦區（推到哪裡都不變）、一大一小是邊緣（只有一個方向會變）、兩者都大才是角點。圖 13-3 的實測也印證了這一點：平坦區的 (λ1, λ2) 是 (3.2, 3.1)，邊緣是 (29120, 105)，比值高達 276 倍，而角點是 (39345, 37386)，兩者都大且比值只有 1.1。

問題是 1988 年算特徵值太貴。Harris 的巧思是：不必真的解出 λ1 與 λ2，只要用它們的兩個對稱函數就夠了——$\det(M) = \lambda1\lambda2$、$\mathrm{trace}(M) = \lambda1 + \lambda2$，而這兩個量可以直接從矩陣元素算出來，不需要開根號。於是定義 $R = \lambda1\lambda2 - k(\lambda1 + \lambda2)^2$：平坦區兩者都小，R 接近零;邊緣時 $\lambda2 \approx 0$，det 接近零而 trace 很大，R 變成大的負值;只有兩者都大時 R 才是大的正值。用一個不必開根號的式子，就把三種情況分開了。

那個 k 呢?教科書一律說取 0.04 到 0.06，但很少說它有一個數學上的上限。把 $\lambda1 = \lambda2 = \lambda$（最理想的角點）代進去：$R = \lambda^2 - k(2\lambda)^2 = \lambda^2(1 - 4k)$。一旦 $k > 0.25$，括號內就變成負的——連最完美的角點都會得到負的響應，整張影像再也找不到任何角點。圖 13-5 的實測正好驗證：k 從 0.01 到 0.15 之間，對旋轉的重複性只在 79.6% 到 82.5% 之間游移，起伏不到 3 個百分點;但 k 推到 0.24 時就崩到 54.7%。

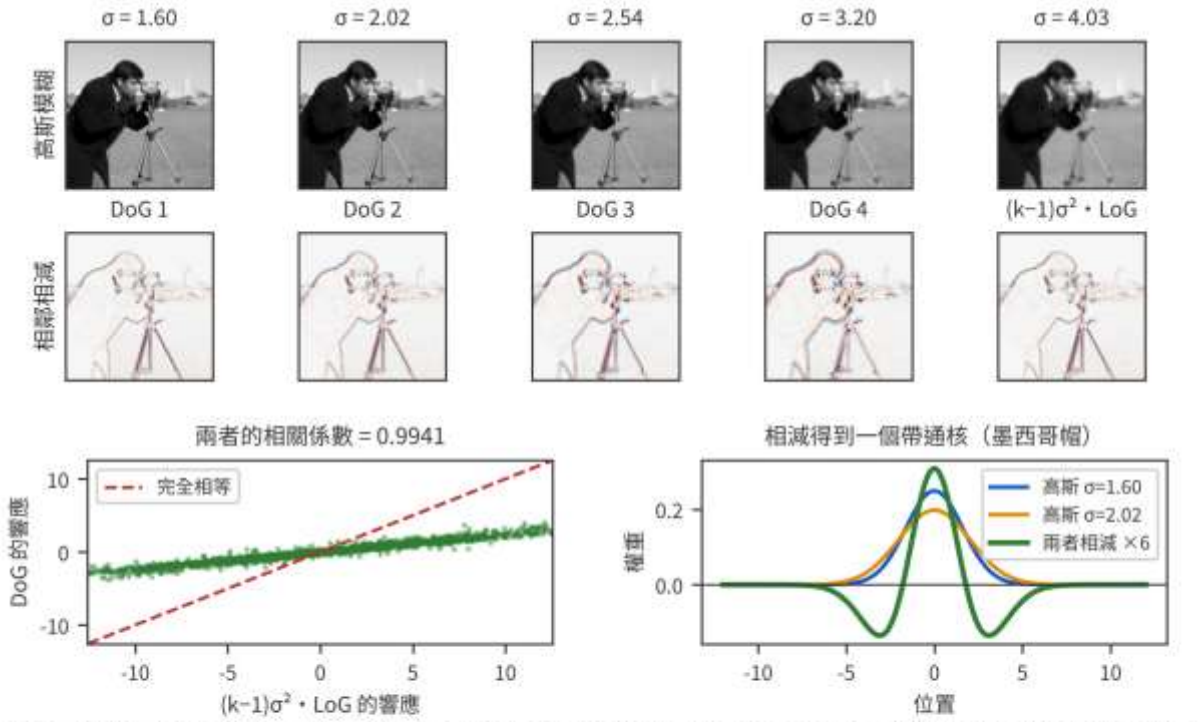
這也說明了 k 真正在控制什麼。它不改變角點的強弱排序（所以「取前 N 個最強的角點」幾乎不受影響），它改變的是「在固定門檻下有多少像素被判為角點」——圖 13-5 右下實測，k 從 0.01 到 0.24，超過門檻的像素從 2,709 個降到 164 個。理解了這一點，就會明白 Shi-Tomasi 的取捨：它直接取 $\min(\lambda1, \lambda2)$，判準的意義一目了

然、沒有隱藏的上限、也沒有參數要調，代價是真的要解一次特徵值。以今天的算力，這個代價早就不值一提——這是演算法設計中很常見的一幕：當年為了省算力而做的近似，在算力變便宜之後，反而成了需要被解釋的負擔。

13.3 尺度不變特徵：SIFT 與 ORB

13.3.1 SIFT 的四個步驟

承叫獸開講，SIFT 的偉大之處在於它同時解決了尺度與旋轉不變性。它的完整流程可以拆成四步，每一步都巧妙地呼應了本書前面的概念：



SIFT 的第一步是建一座尺度金字塔，而它找極值用的不是影像本身，是相鄰兩層的「高斯差」（DoG）。為什麼可以這樣？右下角給了直覺：兩個寬度略不同的高斯相減，得到一個中間正、周圍負的帶通核——也就是第 9 章的墨西哥帽（LoG）。嚴格一點說，熱傳導方程式給出 $\partial G/\partial \sigma = \sigma \cdot \nabla^2 G$，把左邊的微分換成差分就得到 $G(k\sigma) - G(\sigma) \approx (k-1)\sigma^2 \cdot \nabla^2 G$。本圖把兩邊實際算出來比對：相關係數高達 0.9941（左下散點圖）。這個近似為什麼重要？因為 LoG 要做二階微分、運算貴，而高斯模糊無論如何都要做（建金字塔本來就要），相減則幾乎不花錢。SIFT 等於用一個「順手」的運算換到了昂貴的帶通濾波器——這是演算法設計上非常漂亮的一步，也是第 11 章多解析度分析與第 9 章邊緣偵測在本章交會的地方。

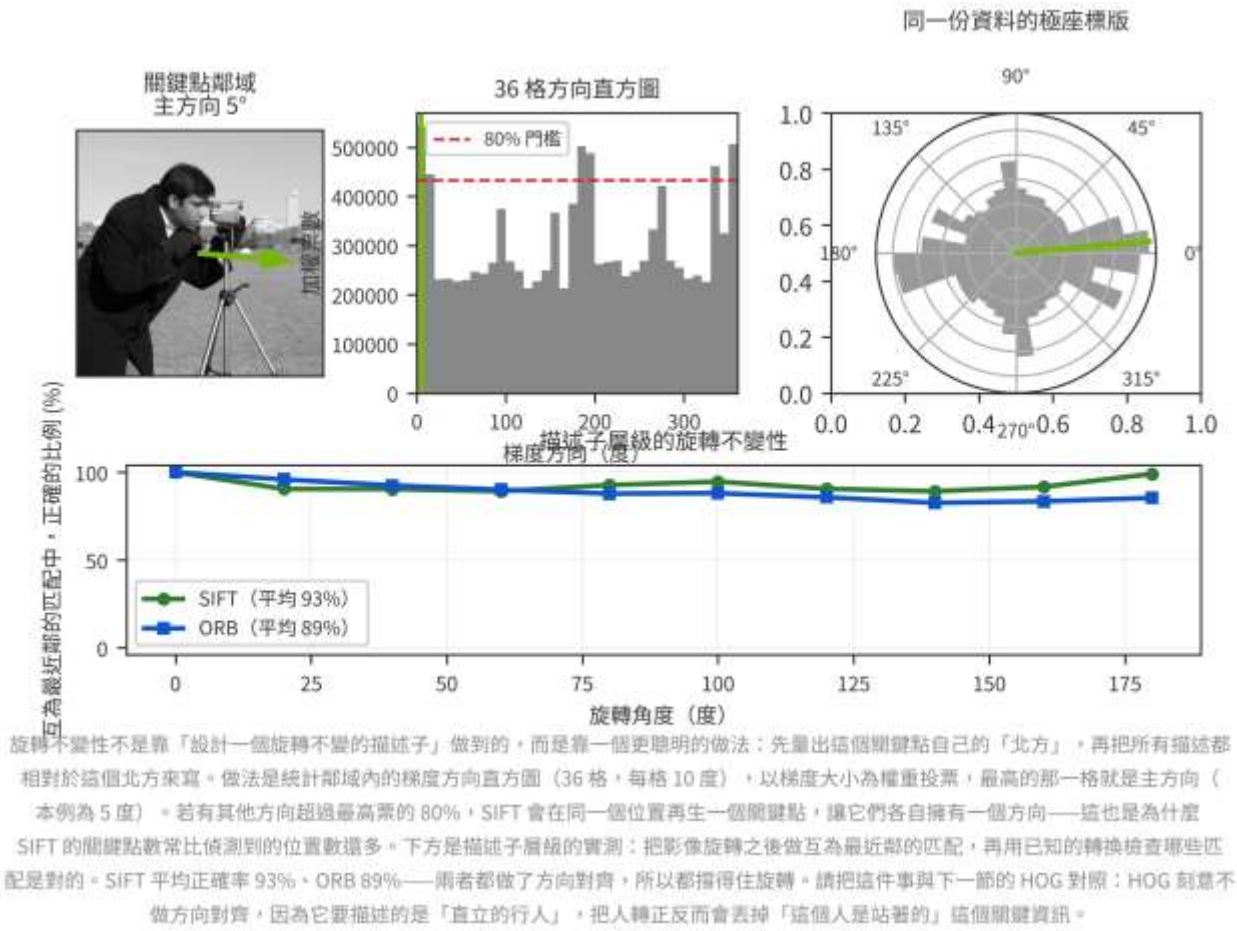
圖 13-7　尺度空間與 DoG：為什麼相減就能近似 LoG

4. 尺度空間極值偵測：把影像用一系列愈來愈強的高斯模糊處理，建構出一個「尺度金字塔」(這正是第 11 章多解析度分析的思想!)。相鄰兩層相減得到「高斯差」(DoG)，它是第 9 章 LoG 的高效近似，也是一個帶通濾波器(第 6 章)。

在這個「空間加尺度」的三維空間中尋找極值點——一個點若在自己的尺度與相鄰尺度中都是局部極值，它就是候選關鍵點。關鍵在於：因為在多個尺度中搜尋，無論物體以什麼大小出現，都能在對應的尺度層被找到，這就實現了尺度不變性。

5. 關鍵點精確定位：剔除對比度太低(易受雜訊干擾)與位於邊緣上(定位不穩)的候選點，留下真正穩定的關鍵點。
6. 方向指派：分析關鍵點周圍的梯度方向分布，把最主要的方向指派給該關鍵點作為「基準方向」。之後所有的描述都相對於這個基準方向來計算——這就實現了旋轉不變性(無論物體轉了多少度，基準方向會跟著轉，描述保持一致)。
7. 描述子生成：在關鍵點周圍取一個區塊，依基準方向對齊後，統計區塊內各子區域的梯度方向直方圖，組成一個 128 維的向量——這就是該關鍵點的「指紋」(描述子)。這個向量經過正規化以抵抗光照變化，最終得到一個對尺度、旋轉、光照都穩健的特徵描述。

值得回味的是，SIFT 幾乎是本書前半的集大成：高斯模糊(第 5 章)、多尺度(第 11 章)、DoG/帶通(第 6、9 章)、梯度(第 9 章)、直方圖(第 5 章)、正規化(第 5 章)——這些看似分散的工具，在 SIFT 中被組織成一個優雅的整體。這也說明了為什麼要循序學完前面各章：它們是理解進階演算法的積木。



旋轉不變性不是靠「設計一個旋轉不變的描述子」做到的，而是靠一個更聰明的做法：先量出這個關鍵點自己的「北方」，再把所有描述都相對於這個北方來寫。做法是統計鄰域內的梯度方向直方圖（36 格，每格 10 度），以梯度大小為權重投票，最高的那一格就是主方向（本例為 5 度）。若有其他方向超過最高票的 80%，SIFT 會在同一個位置再生一個關鍵點，讓它們各自擁有一個方向——這也是為什麼 SIFT 的關鍵點數常比偵測到的位置數還多。下方是描述子層級的實測：把影像旋轉之後做互為最近鄰的匹配，再用已知的轉換檢查哪些匹配是對的。SIFT 平均正確率 93%、ORB 89%——兩者都做了方向對齊，所以都撐得住旋轉。請把這件事與下一節的 HOG 對照：HOG 刻意不做方向對齊，因為它要描述的是「直立的行人」，把人轉正反而會丟掉「這個人是站著的」這個關鍵資訊。

圖 13-8　方向指派：先量出自己的北方，再相對於它來描述

13.3.2 ORB：免費而快速的替代方案

SIFT 雖強，卻有兩個現實問題：受專利保護(叫獸開講的核心)、且運算較慢(128 維浮點描述子，計算與比對都吃資源)，不適合即時或資源受限的場合。2011 年 OpenCV 團隊提出的 ORB，正是為了解決這兩點。

ORB 是兩個既有技術的聰明融合：用 FAST 角點偵測器(極快)找關鍵點，並加上方向資訊使其具旋轉不變性;用 BRIEF 二進位描述子(以一連串「這個點比那個點亮嗎」的是非比較，產生一串 0 與 1)來描述特徵，並加以旋轉校正。二進位描述子的好處是：比對時用漢明距離(算兩串位元有幾位不同)，可以用 CPU 指令極速完成，遠快於 SIFT 的浮點運算。ORB 完全免費、速度可比 SIFT 快數十倍，雖然在尺度變化極大或精度要求極高的場合略遜 SIFT，但在多數即時應用(尤其是機器人、手機、樹莓派 / Jetson 等第 17 章的邊緣平台)中是務實首選。

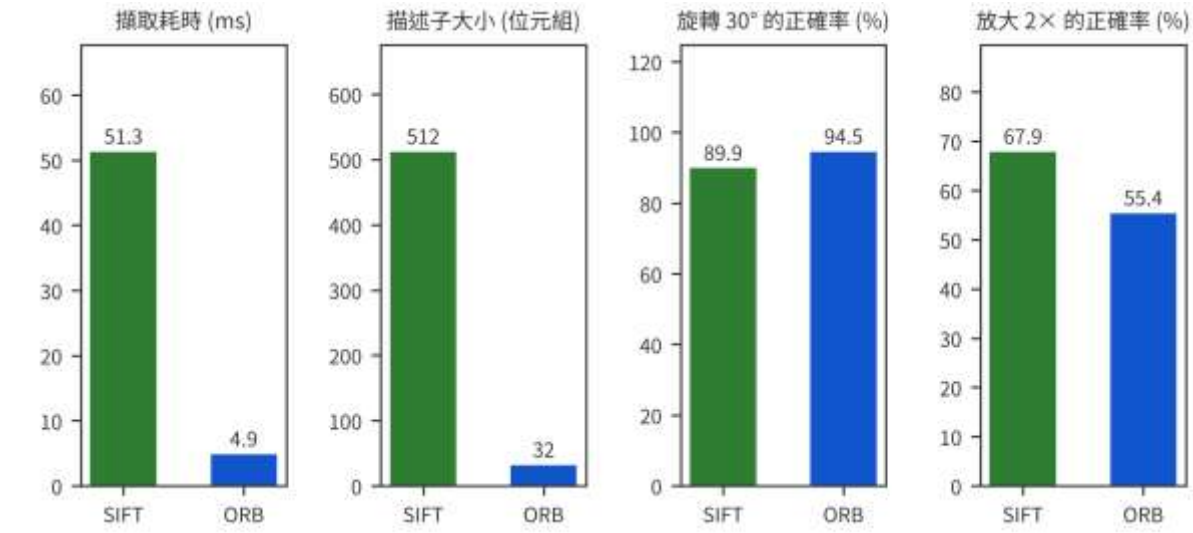


表 13-1 列了 SIFT 與 ORB 的差異，這張圖把它量出來。在同一張 512×512 影像、同樣上限 800 個特徵的條件下：ORB 擷取只要 4.9 毫秒，SIFT 要 51.3 毫秒，快了 10.6 倍;描述子一個是 32 位元組的二進位碼，一個是 512 位元組的浮點向量，差 16 倍。右邊兩格是代價，但結果可能和你預期的不同。旋轉 30 度時 ORB 的匹配正確率 94.5% 反而略高於 SIFT 的 89.9%——ORB 也會先量主方向再旋轉它的取樣樣式（rBRIEF），所以旋轉這一項它並不吃虧，「SIFT 一定比較準」這句話並不成立。真正的分水嶺在尺度：放大 2 倍時 SIFT 有 67.9%，ORB 只剩 55.4%，因為 ORB 的金字塔層數遠少於 SIFT，尺度跨度一大就跟不上（這與上一張圖的結論一致）。所以選擇的依據很清楚：即時系統、嵌入式裝置、尺度變化不大（例如同一台固定相機的連續影格），選 ORB;要跨尺度比對、拼接差異很大的照片、或精度優先，選 SIFT。再加上叫獸閱讀那一課——2020 年之前還要考慮專利，而這正是 ORB 當年能站穩腳步的真正原因。

圖 13-9　SIFT 與 ORB 的四項實測對照

比較面向	SIFT	ORB
授權	原有專利(2020 年到期)	完全免費開源
描述子	128 維浮點向量	二進位字串(漢明距離比對)
速度	較慢	快數十倍，可即時
尺度不變性	優異	中等(尺度變化大時較弱)
適用場合	高精度離線比對、拼接	即時、行動與邊緣裝置

表 13-1 SIFT 與 ORB 的比較

程式 13-2 SIFT 與 ORB 特徵擷取

```
import cv2

img = cv2.imread("object.jpg", cv2.IMREAD_GRAYSCALE)

# --- SIFT(專利已於 2020 到期,可自由使用) ---
sift = cv2.SIFT_create()
kp_sift, des_sift = sift.detectAndCompute(img, None)
print("SIFT 關鍵點數:", len(kp_sift),
      "描述子維度:", des_sift.shape)      # (N, 128)
```

```
# --- ORB(免費、快速) ---
orb = cv2.ORB_create(nfeatures=500)
kp_orb, des_orb = orb.detectAndCompute(img, None)
print("ORB 關鍵點數:", len(kp_orb),
      "描述子維度:", des_orb.shape)        # (N, 32) 二進位

# 畫出關鍵點(含尺度與方向)
vis = cv2.drawKeypoints(img, kp_sift, None,
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imwrite("keypoints.png", vis)
```

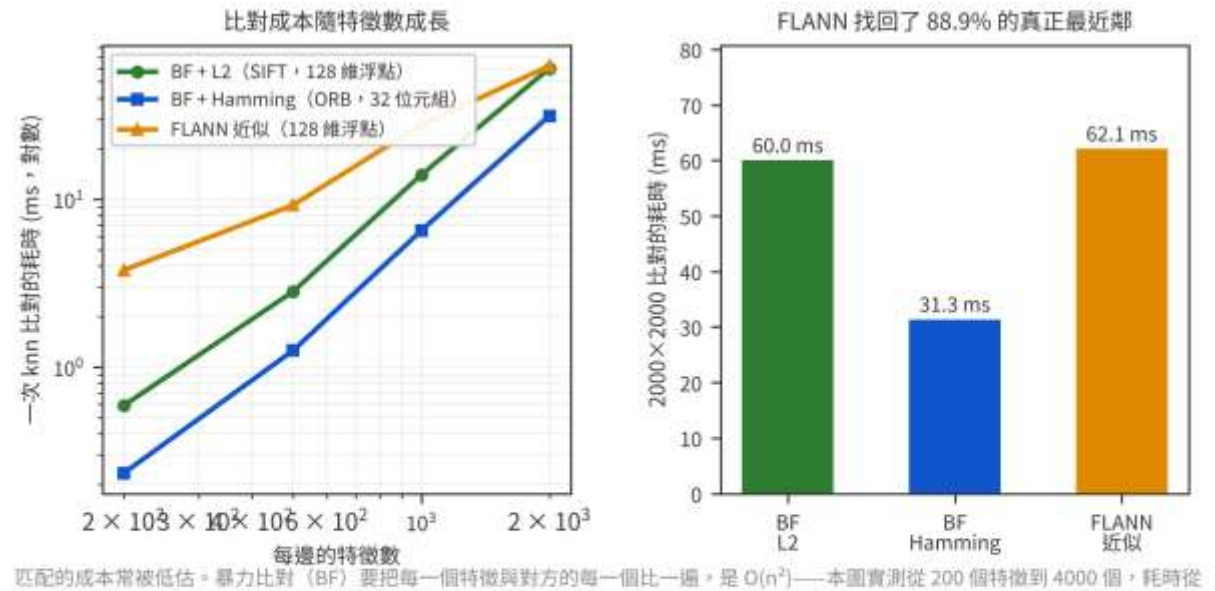
13.4 特徵匹配與外點去除

偵測出兩張影像各自的特徵與描述子後，下一步是匹配(Matching)：找出「這張影像的哪個特徵，對應到那張影像的哪個特徵」。這是影像拼接、物件辨識、3D 重建的關鍵一步。

13.4.1 匹配器：BF 與 FLANN

最直接的方法是暴力匹配(Brute-Force,BF)：把第一張影像的每個描述子，與第二張影像的所有描述子逐一比對距離，取最近的作為匹配。距離的計算方式取決於描述子類型——SIFT 的浮點描述子用歐氏距離，ORB 的二進位描述子用漢明距離。BF 準確但當特徵量大時很慢。

FLANN(快速近似最近鄰函式庫)則以近似的方式加速搜尋：它建立特殊的資料結構(如 KD 樹)，不保證找到絕對最近的匹配，但速度快得多，適合大量特徵的場合。BF 與 FLANN 的取捨，又是一次「準確 vs. 速度」的工程權衡——與第 9 章門檻、第 10 章偵測參數、第 12 章追蹤器選擇一脈相承。

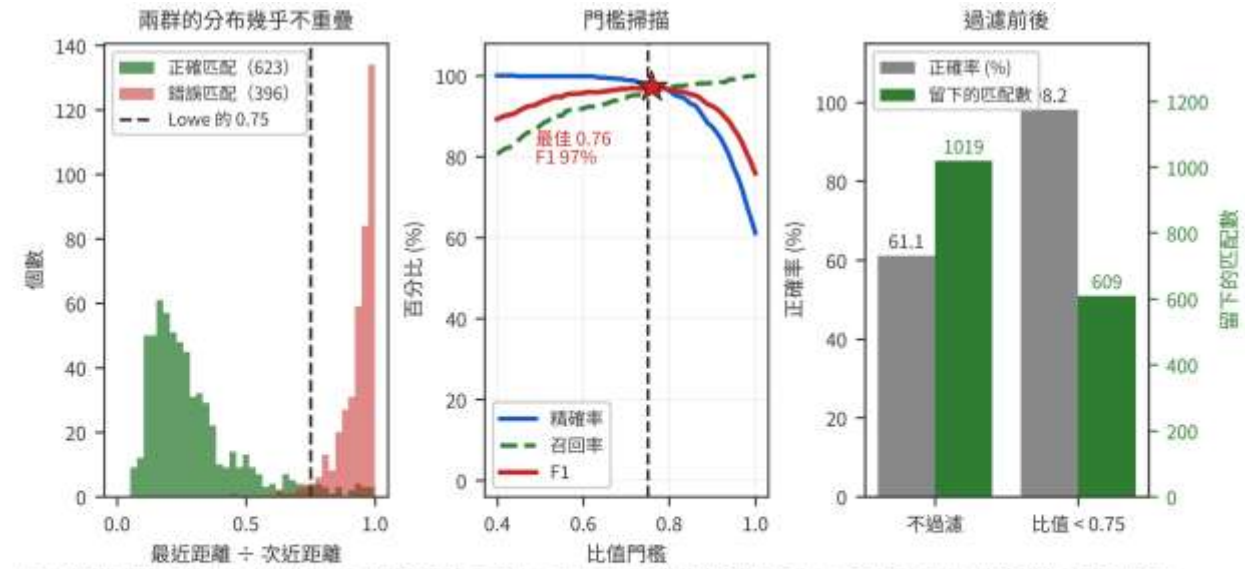


匹配的成本常被低估。暴力比對（BF）要把每一個特徵與對方的每一個比一遍，是 $O(n^2)$——本圖實測從 200 個特徵到 4000 個，耗時從 0.6 毫秒漲到 60 毫秒。ORB 的二進位描述子在這裡展現真正的威力：Hamming 距離只是互斥或再數 1，CPU 有專門的指令，2000×2000 時只要 31.3 毫秒，而 L2 要 60.0 毫秒，快了 2 倍。注意這已經包含了「32 位元組對 512 位元組」的資料量差距，兩個因素是疊加的。FLANN 走另一條路：用 kd 樹做近似搜尋，2000×2000 時 62.1 毫秒。但它是近似的——實測在真實的 SIFT 描述子上，它找回了 88.9% 的真正最近鄰。（這個實驗一定要用真實描述子：隨機的高維向量彼此幾乎等距，近似搜尋的召回率會低到毫無代表性，量到的是維度災難而不是 FLANN 的本事。）觀念要記住：FLANN 用一點正確率換速度，而且特徵數愈多愈划算;精度優先、特徵數又不多的場合，仍該用 BF。

圖 13-10　比對的成本：BF、漢明距離與 FLANN 近似搜尋

13.4.2 比值測試：過濾模糊匹配

原始匹配充滿錯誤——因為並非每個特徵都在另一張影像中有真正的對應(可能被遮擋、或根本不存在)。David Lowe 在 SIFT 論文中提出了一個極為有效的過濾法：比值測試(Ratio Test)。對每個特徵，找出它在另一張影像中最近與次近的兩個匹配;若「最近距離」明顯小於「次近距離」(例如小於 0.75 倍)，代表這個匹配獨一無二、可信;若兩者接近，代表這個特徵在對方影像中有多個相似的候選、難以區分，應該捨棄。這個簡單的比值，能濾掉大量不可靠的匹配。



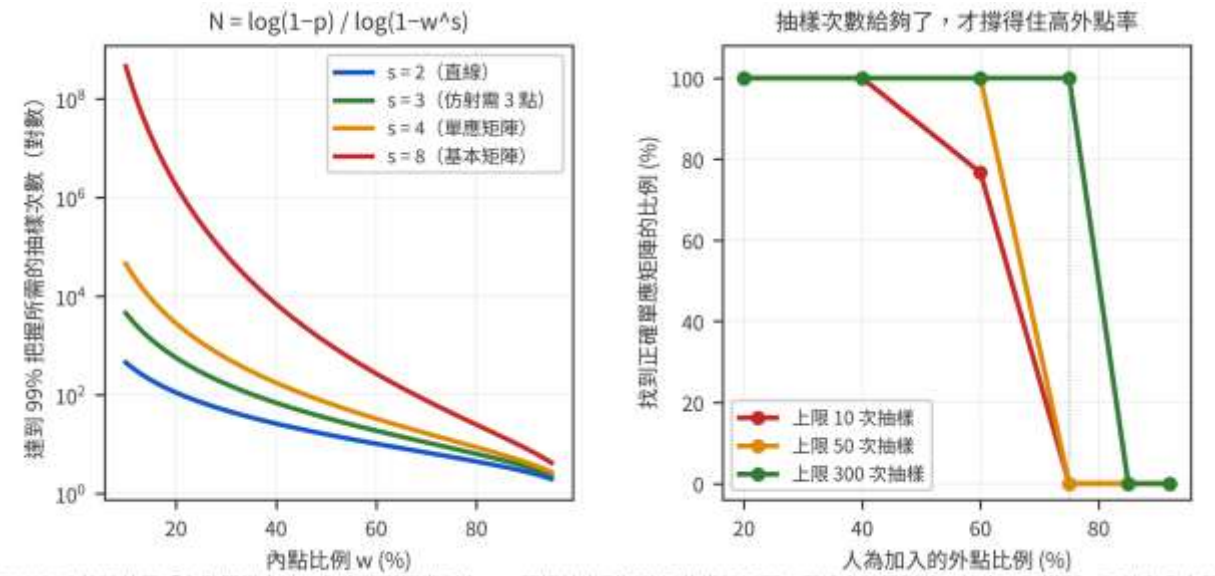
Lowe 比值測試的巧妙之處，在於它不看「最近的距離有多近」，而看「最近的比次近的近多少」。理由是：如果一個特徵在對方影像中真的有唯一的對應，那麼最近鄰應該明顯勝過第二名;若第一名與第二名難分軒輊，代表這個特徵長得很普通（例如重複的紋理、磚牆、樹葉），配哪一個都不可靠。左圖是實測的證據：本圖用已知的單應矩陣造出一對影像，所以每一個匹配是對是錯都查得出來。兩群的分布幾乎不重疊——錯誤匹配的比值大多擠在 0.9 以上。中圖掃描門檻：最佳值在 0.76（F1 97%），而 Lowe 論文建議的 0.75 給出精確率 98.2%、召回率 96.0%，相當接近最佳。右圖是這一刀的效果：原始 1019 個匹配中只有 61.1% 是對的，過濾後剩 609 個，正確率升到 98.2%。一個不用訓練、只比兩個數字的規則就能做到這件事，這是整個電腦視覺裡性價比最高的技巧之一。

圖 13-11　比值測試：為什麼要看「第二名」

13.4.3 RANSAC：在錯誤中找出真相

即使經過比值測試，仍會有一些「看起來可信、實則錯誤」的匹配(稱為外點，outliers)。RANSAC(隨機抽樣一致法)是對抗外點的利器，其思想極為漂亮：與其相信所有資料，不如反覆隨機抽取少數幾組匹配，假設它們是正確的、據此計算出一個幾何模型(例如第 4 章的透視轉換矩陣)，再看有多少其他匹配符合這個模型;重複多次，「獲得最多支持者」的模型就是正確答案，而不符合它的匹配就是外點，予以剔除。

RANSAC 的精神與第 9 章霍夫轉換異曲同工——都是「投票」：霍夫讓每個邊緣點投票給可能的直線，RANSAC 讓每個匹配投票給可能的幾何模型，得票最高者勝出。這種「用多數證據對抗少數雜訊」的思想，是穩健估計的核心，在充滿雜訊與錯誤的真實資料中至關重要。



RANSAC 的精神是「少數服從多數，但先隨機問幾個人」：隨機抽出剛好夠用的幾組匹配、假設它們是對的、算出一個模型，再數有多少其他匹配同意這個模型;重複多次，票最高的勝出。要抽幾次才夠?這有閉式解。若內點比例是 w、建一個模型最少要 s 組匹配，那麼一次抽樣全中的機率是 w^s，連抽 N 次都沒全中的機率是 $(1-w^s)^N$，要讓成功機率達到 p，就需要 $N = \log(1-p) / \log(1-w^s)$。左圖把這條式子畫出來，重點在縱軸是對數：單應矩陣要 4 組點（s = 4），內點 50% 時只要 71 次，內點 20% 時要 2876 次，內點 10% 時暴增到 46049 次。右圖是實測，而且刻意把抽樣次數的上限調低，好讓這條公式吃得到苦頭。上限 10 次只撐得到外點 40%、上限 50 次只撐得到外點 60%、上限 300 次只撐得到外點 75%。三條曲線崩潰的位置，正好對應左圖那條理論曲線穿過各自上限的地方——外點 60% 時理論需要 178 次，所以上限 10 次的那條已經開始掉（77%）;外點 85% 時理論需要 9094 次，三條就全軍覆沒了。兩個實務上的推論：一是 s 要愈小愈好（這是為什麼能用仿射就別用單應）;二是先做比值測試把內點比例拉高，RANSAC 會輕鬆非常多——這兩步是搭配使用的，不是二選一。

圖 13-12　RANSAC 要抽幾次才夠：一條有閉式解的公式

程式 13-3 特徵匹配、比值測試與 RANSAC 物件定位

```
import cv2, numpy as np

img1 = cv2.imread("box.jpg", cv2.IMREAD_GRAYSCALE)        # 要找的物件
img2 = cv2.imread("scene.jpg", cv2.IMREAD_GRAYSCALE)      # 場景

sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

# --- BF 匹配 + 比值測試 ---
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)     # 取最近兩個
good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:     # Lowe 比值測試
        good.append(m)
print("比值測試後的匹配數:", len(good))

# --- RANSAC 求單應性矩陣,剔除外點 ---
if len(good) > 10:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1,1,2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1,1,2)
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    print("RANSAC 判定的內點數:", int(mask.sum()))
```

```
# 在場景中畫出物件的位置(第 4 章透視轉換)
h, w = img1.shape
corners = np.float32([[0,0],[0,h-1],[w-1,h-1],[w-1,0]]).reshape(-1,1,2)
projected = cv2.perspectiveTransform(corners, H)
scene = cv2.cvtColor(img2, cv2.COLOR_GRAY2BGR)
cv2.polylines(scene, [np.int32(projected)], True, (0,255,0), 3)
cv2.imwrite("object_located.png", scene)
```

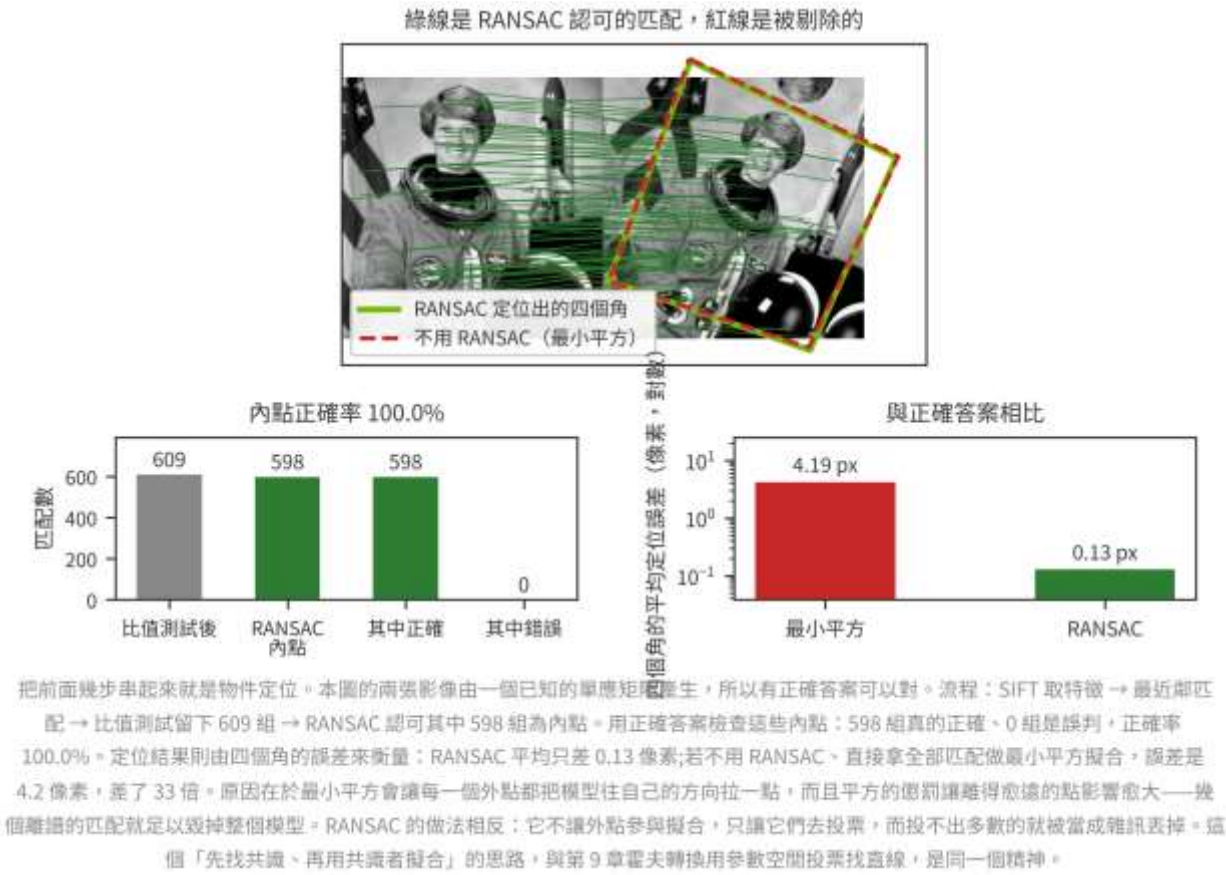


圖 13-13　RANSAC 實戰：用了與不用，定位誤差差三十倍

13.4.4 推導：RANSAC 要抽幾次才夠

RANSAC 是隨機演算法，所以它不保證成功，只保證「成功的機率」。好消息是這個機率可以精確算出來，而且算式很短，值得親手推一次。

設內點（正確匹配）佔全部匹配的比例為 w，而建立一個模型最少需要 s 組匹配（直線要 2 點、仿射轉換要 3 點、單應矩陣要 4 點、基本矩陣要 8 點）。一次隨機抽樣時，抽到的 s 組全是內點的機率是 w^s（匹配數夠多時可視為獨立）。那麼「這一次沒有全中」的機率是 1 – w^s，「連抽 N 次都沒有全中」的機率就是 (1 – w^s)^N。我們希望至少成功一次的機率達到 p（例如 0.99），於是 1 – (1 – w^s)^N ≥ p，兩

邊取對數整理即得 N ≥ log(1 – p) / log(1 – w^s)。

這條式子的性質值得細看，因為它決定了實務上的每一個選擇。第一，N 對 w 極度敏感，而且是指數式的。以單應矩陣（s = 4、p = 0.99）為例：內點 90% 時只要 4 次，內點 50% 時要 71 次，內點 30% 時要 566 次，內點 20% 時要 2,876 次，內點 10% 時暴增到 46,049 次。圖 13-12 左圖把這條曲線畫出來，縱軸必須用對數才畫得下。

第二，N 對 s 也極度敏感。同樣在內點 50% 的情況下，s = 2 只要 16 次、s = 4 要 71 次、s = 8 則要 1,177 次。這給了一條很實用的原則：能用自由度低的模型就別用高的。若已知兩張影像只差旋轉與平移，就別去解 8 個自由度的單應矩陣，用 4 個自由度的相似轉換，RANSAC 會輕鬆非常多，而且不容易被外點誘導出一個「扭曲但碰巧符合」的解。

第三，這條式子也解釋了比值測試為什麼必須先做。圖 13-11 的實測：原始匹配的正確率只有 61.1%，經過比值測試之後升到 98.2%。代進公式，w 從 0.61 變成 0.98，所需抽樣次數從 29 次降到 2 次。比值測試與 RANSAC 不是二選一，而是一前一後的搭配——前者便宜地把 w 拉高，後者才有辦法在合理的次數內收斂。

實務上還有一個細節：真正的實作（包括 OpenCV）不會把 N 寫死，而是「邊做邊更新」。每找到一個更好的模型，就用它的內點數重新估計 w，再據此下修 N。所以你設的 maxIters 其實是一個上限，不是實際次數。圖 13-12 右圖刻意把這個上限調低，好讓理論吃得到苦頭：上限 10 次時外點 60% 就開始失手（成功率 77%），上限 50 次能撐到外點 60%，上限 300 次則撐到外點 75%。三條曲線崩潰的位置，正好對應理論曲線穿過各自上限的地方。這也提醒一件事——當你的 RANSAC 常常失敗，該先檢查的是內點比例，而不是一味調高迭代次數。

13.5 區域描述子與形狀特徵：HOG、LBP 與幾何量測

到這裡為止，本章的特徵都屬於同一種類型：在影像中找出少數幾個「點」，為每個點生成描述子，再跨影像配對這些點。這類關鍵點特徵(keypoint features)回答的是一個很特定的問題——「這兩張影像裡的哪一點是同一點?」它是拼接、定位、追蹤與 SLAM 的基礎。但真實世界還有另一大類問題：「這一塊區域是什麼?」這是行人嗎?這片金屬表面有刮痕嗎?這個零件的外形合格嗎?此時我們根本不需要知道點與點的對應關係，而需要一個能把整塊區域濃縮成一串數字、並且能直接餵給分類器的描述。

本節補上這一塊：三種最重要的區域與形狀描述子——HOG、LBP 與以輪廓為基礎的幾何特徵。它們既是深度學習之前電腦視覺的主力，也是第 14 章傳統機器學習的標準輸入;更重要的是，前面第 8、9、10 章多次預告的「紋理特徵」與「形狀特徵」，答案都在這裡。

13.5.1 兩種特徵觀：對應問題與辨識問題

關鍵點特徵與區域描述子的差別，可以從三個角度看清楚。

8. 取樣是稀疏還是稠密:關鍵點特徵是稀疏(sparse)的——它只在影像中「值得注意」的少數位置取值，一張照片可能只產生幾百到幾千個關鍵點，其餘區域一概不管。區域描述子則是稠密(dense)的——它在一個固定視窗內逐格取樣、全面覆蓋，不挑位置，因為我們要描述的是「整塊」而非「某點」。
9. 是否需要配對:關鍵點特徵的價值在配對，所以必須先經過匹配、比值測試與 RANSAC(13.4 節)。區域描述子不配對，而是把向量直接交給分類器(第 14 章的 SVM、kNN)，或是用直方圖距離比較兩塊區域像不像。
10. 描述的對象是灰階分布還是幾何:HOG 與 LBP 描述的仍是視窗內的灰階/梯度分布;形狀特徵則更進一步，描述一個「已經被切出來的物件」的幾何性質——它的輸入不是灰階影像，而是第 9 章分割、第 10 章型態學清理與連通元件標記之後得到的二值遮罩或輪廓。

這個分類也解釋了為什麼本節要放在這個位置：關鍵點特徵處理「對應問題」，區域與形狀特徵處理「辨識問題」，而一條完整的工業視覺管線，兩者往往都要用上。



13.5 節開頭說本章有「兩種特徵觀」，這張圖把差別畫出來。左邊是前四節的世界：偵測器在影像中挑出少數值得注意的位置，本例 241 個關鍵點，其餘區域一概不管。中間是接下來的世界：不挑位置，把視窗切成固定的網格，本例 32×32 = 1024 個細胞，每一格都要算、每一格都算進最後的向量。差別不只是「挑不挑位置」，而是要解的問題根本不同。關鍵點特徵要解的是對應問題——這張照片的這個點，是不是那張照片的那個點?所以它必須配對，也因此需要比值測試與 RANSAC。區域描述子要解的是辨識問題——這塊區域裡有沒有行人?它不配對，而是把一個長度固定的向量直接交給分類器（第 14 章）。長度固定這件事很關鍵：分類器要求每個樣本的維度一致，而關鍵點的數量每張影像都不同，沒辦法直接餵。一個 64×128 的行人視窗，HOG 給出的就是固定的 3,780 維。

圖 13-14　兩種特徵觀：稀疏取樣找對應，稠密取樣做辨識

13.5.2 HOG：方向梯度直方圖

2005 年，法國 INRIA 研究院的 Navneet Dalal 與 Bill Triggs 在 CVPR 發表了方向梯度直方圖(Histogram of Oriented Gradients,HOG)。在深度學習興起之前，「HOG + SVM」幾乎就是行人偵測的同義詞，也是汽車主動安全系統最早能真正上路的視覺技術之一。

HOG 的核心洞見只有一句話：一個物體的外形，主要寫在「邊緣方向的分布」裡，而不在絕對亮度裡。一個直立的人，不論穿深色或淺色衣服、不論在陽光下或陰影中，他的輪廓都會呈現同一套方向模式——肩線偏水平、軀幹兩側偏垂直、雙腿之間有一道縱向間隙。只要把這套方向模式量化下來，就抓住了「人形」。

HOG 的計算分成四步，每一步都用到本書前面的工具：

11. 梯度計算:先用第 9 章的 Sobel 算子求每個像素的梯度大小與方向。注意 HOG 一般使用「無號方向」，把 0 到 360 度摺疊到 0 到 180 度——因為我們在意的是邊緣的走向，而不在意它是由亮到暗還是由暗到亮。
12. 細胞直方圖:把影像切成不重疊的小細胞(cell,典型為 8×8 像素)。每個細胞內的所有像素，依自己的方向投票到 9 個方向區間(每區間 20 度)，投票的權重是該像素的梯度大小——邊緣愈強，發言權愈大。這一步正是第 5 章直方圖思想的再次運用：用統計分布取代逐點數值，因此對細胞內的小幅位移與變形天生容忍。
13. 區塊正規化:把相鄰的 2×2 個細胞合成一個區塊(block,即 16×16 像素)，對區塊內 4 個直方圖串成的 36 維向量做正規化(常用 L2-Hys:L2 正規化後裁切再正規化)。這一步是 HOG 強健性的關鍵——局部正規化讓描述子對光照強弱與對比變化幾乎無感，呼應第 5 章「先正規化再比較」的原則。區塊以半個區塊(8 像素)為步幅滑動，彼此重疊，讓同一個細胞在不同的正規化脈絡下被重複描述，進一步提升穩定度。
14. 串接成向量:把所有區塊的正規化向量首尾相接，就是整個視窗的 HOG 描述子。

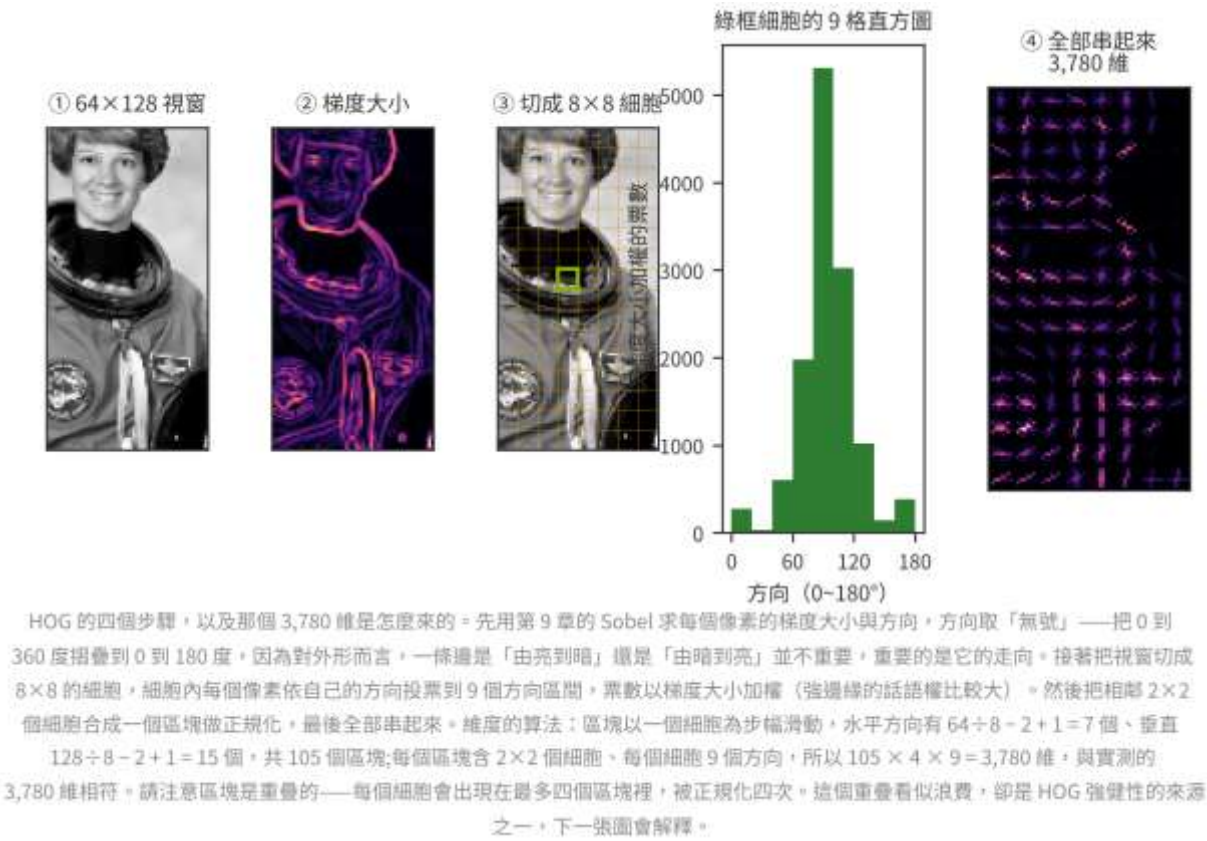


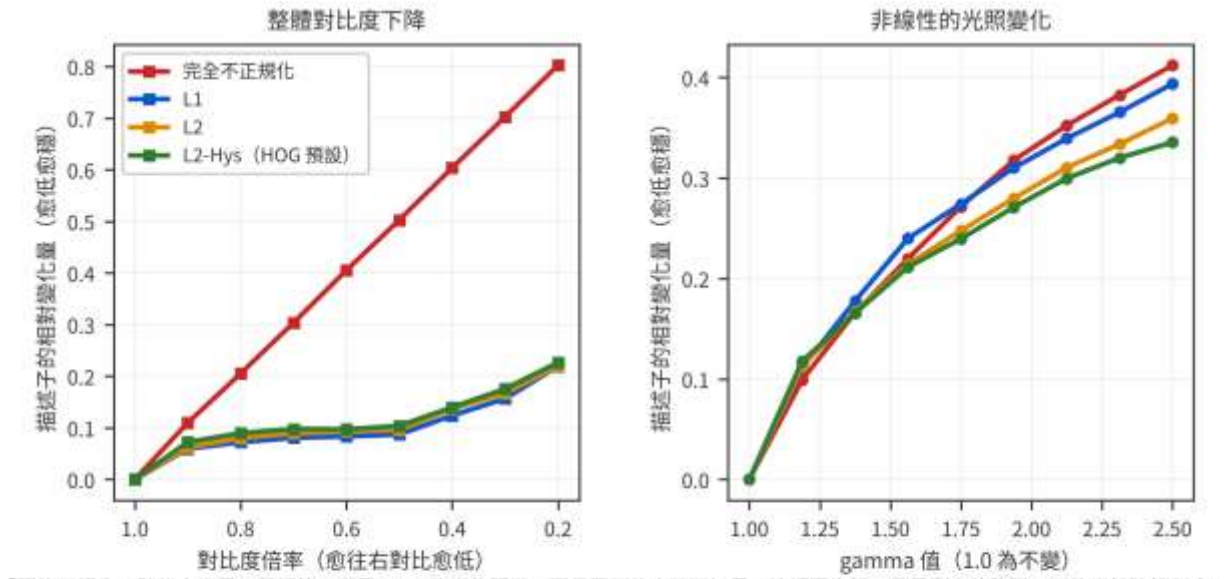
圖 13-15　HOG 的四個步驟，與那個 3,780 維是怎麼來的

以行人偵測的標準視窗 64×128 像素計算：水平方向可放 64÷8−2+1 = 7 個區塊，垂直方向 128÷8−2+1 = 15 個區塊，共 105 個區塊;每區塊 4 個細胞、每細胞 9 個方向，於是維度為 105×4×9 = 3,780 維。這個數字值得記住——它說明了手工特徵的代價：為了描述一個小小的 64×128 視窗，我們產生了近四千維的向量，而且這套規則是人訂的，換一個任務就得重新調整。

HOG 與 SIFT 的關係很值得玩味：兩者骨子裡都是「梯度方向直方圖」。差別在於 SIFT 在關鍵點上稀疏取樣，並且把直方圖對齊到關鍵點的主方向以取得旋轉不變性;HOG 在固定視窗上稠密取樣，而且刻意不做方向對齊——因為行人就是直立的，方向資訊本身就是重要線索，對齊反而會把它丟掉。這也直接說明了 HOG 的侷限：它不具旋轉不變性，而尺度問題則靠「影像金字塔 + 滑動視窗」在外層解決(第 11 章的多解析度思想)。

完整的 HOG 偵測器長這樣：建立影像金字塔，在每一層用固定大小的視窗逐格

滑動，對每個視窗算 HOG 並丟給一個線性 SVM(第 14 章)判斷「是/不是行人」，最後以非極大值抑制合併重疊的框。這條管線的每一個零件，在第 16 章的 YOLO 裡都還在——只是「人工設計的 HOG + SVM」被換成了「自動學出的卷積特徵 + 神經網路輸出頭」，而且不再需要滑動視窗。理解了 HOG，再看 YOLO 就不會覺得它是魔法。



「區塊正規化」聽起來只是工程細節，卻是 HOG 能用的關鍵，而且買到的東西可以量。這裡要先說一個量測上的陷阱：不能用餘弦相似度，因為餘弦對整體縮放免疫，而「對比度下降」正好就是整體縮放，用餘弦會看不見差別（本圖初稿就是這樣得到「正規化沒什麼用」的錯誤結論）。改用相對 L2 距離就清楚了。左圖把對比度壓到 0.2 倍：完全不正規化的描述子整個縮小了 80%，而 L2-Hys 只變動 22.6%，差了 3.5 倍。道理很直接——對比度下降讓所有梯度等比例變小，直方圖的形狀沒變、只是整體縮小；正規化把每個區塊除以自己的長度，等比例的縮放就被除掉了。右圖是比較難的非線性 gamma 變化，不正規化變動 41%、L2-Hys 34%，正規化仍有幫助但無法完全消除。L2-Hys 比 L2 多做一件事：先 L2 正規化、把超過 0.2 的分量截掉、再正規化一次，用意是壓制少數極強的邊緣（例如衣服上的高光）獨佔整個區塊。至於區塊為什麼要重疊——同一個細胞在不同區塊裡會被不同的鄰居正規化，等於用好幾種局部脈絡各描述它一次，單一區塊的異常就不會毀掉它。

圖 13-16　區塊正規化買到了什麼：對比度一降就見真章

13.5.3 LBP：區域二值模式與紋理描述

如果 HOG 描述的是「外形」，那麼區域二值模式(Local Binary Pattern,LBP)描述的就是「表面」。它由芬蘭奧盧大學的 Timo Ojala 等人於 1996 年提出、2002 年在 IEEE TPAMI 上完成理論化，是紋理分析領域影響最深遠的方法之一，至今仍是表面瑕疵檢測(AOI)最具性價比的工具。

LBP 的做法樸素到令人意外：取一個 3×3 鄰域，以中心像素的值當門檻，八個鄰居只問一個問題——「你比中心亮嗎?」亮(大於或等於)記 1，暗記 0。依固定順序把

八個答案串成一個八位元二進位數，就得到 0 到 255 之間的一個碼。整張影像每個位置都算一次，再統計這些碼的直方圖，這個直方圖就是該區域的紋理描述。

這個樸素的設計帶來一個極為珍貴的性質：對單調遞增的灰階變換完全不變。因為 LBP 只比較大小、不使用數值差，只要影像的灰階經過任何「愈亮仍愈亮」的對應(例如 Gamma 校正、線性調亮、整體降對比)，每一組比較的結果都不會翻轉，LBP 碼一模一樣。本書以嚴格遞增的灰階對應表實測驗證：變換前後的 LBP 影像完全相同;即使改用會產生併階的直方圖等化，仍有 99.2% 的像素碼不變。這正是 LBP 在光照多變的產線與戶外場景下格外可靠的原因。

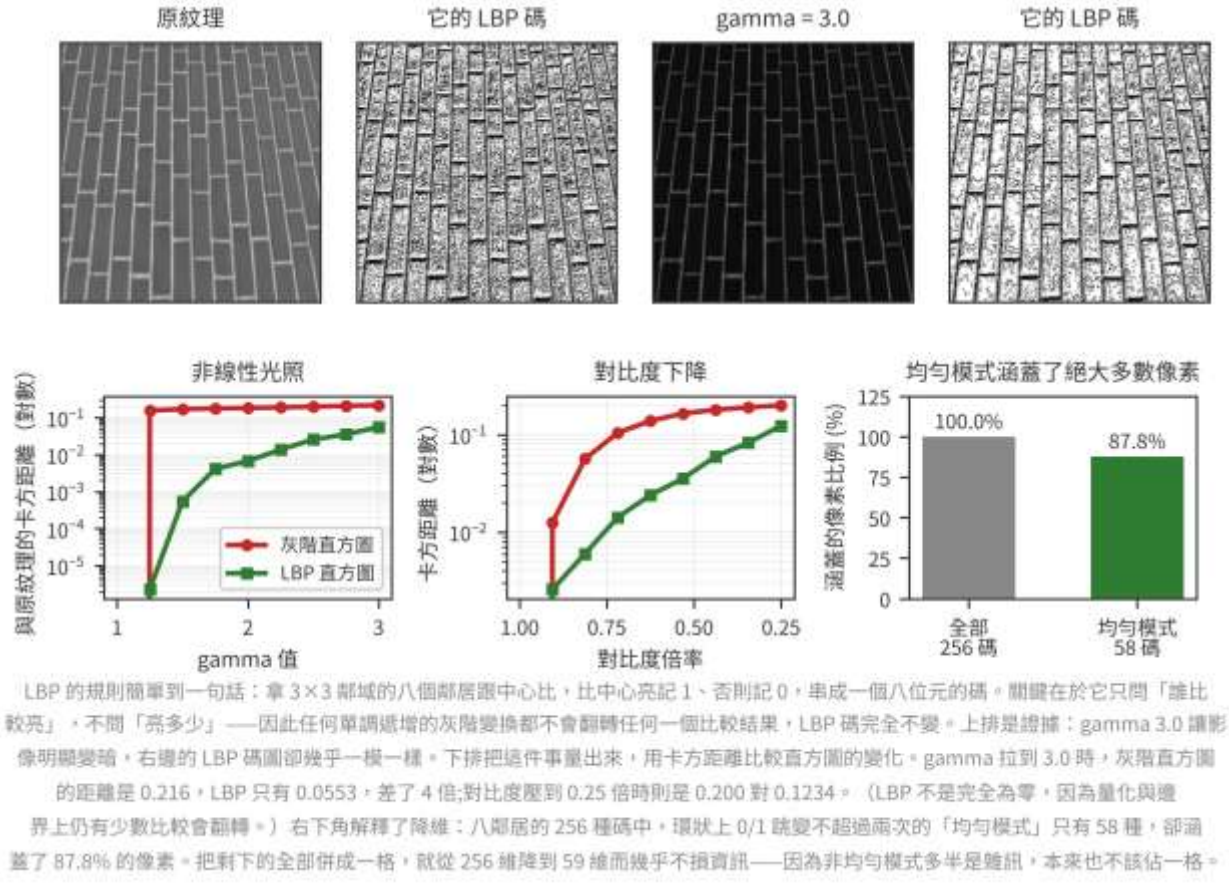
256 維的直方圖偏大，而且其中許多碼其實很罕見。Ojala 等人進一步發現，把八個位元看成一個環，若環上「0 與 1 的跳變次數」不超過兩次，稱為均勻模式(uniform pattern)。直覺上，均勻模式對應的正是自然影像裡真正有意義的局部結構——平坦面(全 0 或全 1)、邊緣、線端、角點;而跳變頻繁的碼多半來自雜訊。在八鄰居的情況下，256 個碼中共有 58 個均勻碼(僅佔 22.7%)，卻能覆蓋自然紋理中約九成的像素。於是標準做法是：58 個均勻碼各給一個區間、其餘全部併為一個「非均勻」區間，共 59 維——維度降到不足四分之一，資訊幾乎不損。

LBP 還有兩個常見的延伸。其一是旋轉不變 LBP：把八位元的環往同一方向循環位移，取所有位移中數值最小者作為代表，於是整塊紋理轉動時碼不變(八鄰居時可壓到 10 維)。其二是多尺度 LBP(P,R)：改成在半徑 R 的圓周上等距取 P 個點，非整數座標處用第 4 章的雙線性內插取值;把 LBP(8,1)、LBP(16,2)、LBP(24,3) 的直方圖串接起來，就能同時描述細、中、粗三種尺度的紋理。

與第 6 章的 Gabor 濾波器對照，可以看出兩條不同的路：Gabor 從頻率域出發，用「特定頻率 + 特定方向」的帶通濾波器去量測紋理的週期與走向;LBP 從空間域出發，用局部大小比較去量測紋理的微結構。Gabor 的物理意義漂亮但運算昂貴(一組濾波器往往要數十次卷積)，LBP 只有比較與查表，便宜到可以在最廉價的嵌入式處理器上即

時執行——這也是第 17 章在談邊緣部署時值得記住的取捨。

在應用上，LBP 的經典戰場有三個：表面瑕疵檢測(把工件表面切成網格，逐格算 LBP 直方圖，與良品樣本比較距離，異常格即為可疑瑕疵)、材質與紋理分類(布料、木紋、路面)，以及人臉辨識(把人臉切成若干區塊、各算直方圖再串接，是深度學習之前的主流方法之一)。第 14 章的農產品與零件分級案例所說的「以紋理特徵判斷表面瑕疵」，用的就是 LBP。



LBP 的規則簡單到一句話：拿 3×3 鄰域的八個鄰居跟中心比，比中心亮記 1、否則記 0，串成一個八位元的碼。關鍵在於它只問「誰比較亮」，不問「亮多少」——因此任何單調遞增的灰階變換都不會翻轉任何一個比較結果，LBP 碼完全不變。上排是證據：gamma 3.0 讓影像明顯變暗，右邊的 LBP 碼圖卻幾乎一模一樣。下排把這件事量出來，用卡方距離比較直方圖的變化。gamma 拉到 3.0 時，灰階直方圖的距離是 0.216，LBP 只有 0.0553，差了 4 倍;對比度壓到 0.25 倍時則是 0.200 對 0.1234。（LBP 不是完全為零，因為量化與邊界上仍有少數比較會翻轉。）右下角解釋了降維：八鄰居的 256 種碼中，環狀上 0/1 跳變不超過兩次的「均勻模式」只有 58 種，部涵蓋了 87.8% 的像素。把剩下的全部併成一格，就從 256 維降到 59 維而幾乎不損資訊——因為非均勻模式多半是雜訊，本來也不該佔一格。

圖 13-17　LBP 的光照不變性：只問誰比較亮，不問亮多少

13.5.4 形狀特徵：從輪廓到 Hu 不變矩

第三類特徵的前提與前兩類不同：它假設物件已經被切出來了。輸入是第 9 章門檻化或分割、再經第 10 章型態學清理與連通元件標記之後得到的二值遮罩或輪廓。既然已經知道「物件在哪」，接下來要問的就是「它長什麼樣」。

最基本的量測有三個：面積 A(遮罩內的像素數)、周長 P(輪廓的長度)，以及外接

矩形的寬 w 與高 h。但這三個量都帶著單位、都隨拍攝距離改變，不能直接拿來比較。真正好用的是由它們組合出來的無因次比值——因為比值會把尺度約掉，天生就對距離遠近與影像縮放不敏感：

15. 圓形度(circularity)4πA/P²:衡量輪廓有多「緊緻」。完美圓形為 1.000，正方形為 0.785，3:1 的長矩形為 0.589;值愈小，代表輪廓愈狹長或愈曲折。這是區分「圓孔/圓形零件」與「長條狀物件」最快的一刀。
16. 長寬比(aspect ratio)w/h:最直觀的外形指標。13.7 節車牌偵測用「約 3:1 到 4:1」篩選候選區域，用的就是這個量。
17. 填充率(extent)A/(w·h):物件填滿其外接矩形的比例。實心矩形接近 1，十字形或 L 形零件則明顯偏低。
18. 凸實度(solidity)A/A 凸包:物件面積與其凸包面積之比。完整的凸形物件接近 1，一旦邊緣有缺角、崩邊或凹口就會明顯下降——這使它成為瑕疵檢測中最便宜的一道判準。
19. 等效直徑 √(4A/π):與該物件同面積的圓的直徑，便於把不規則物件換算成一個「代表尺寸」。

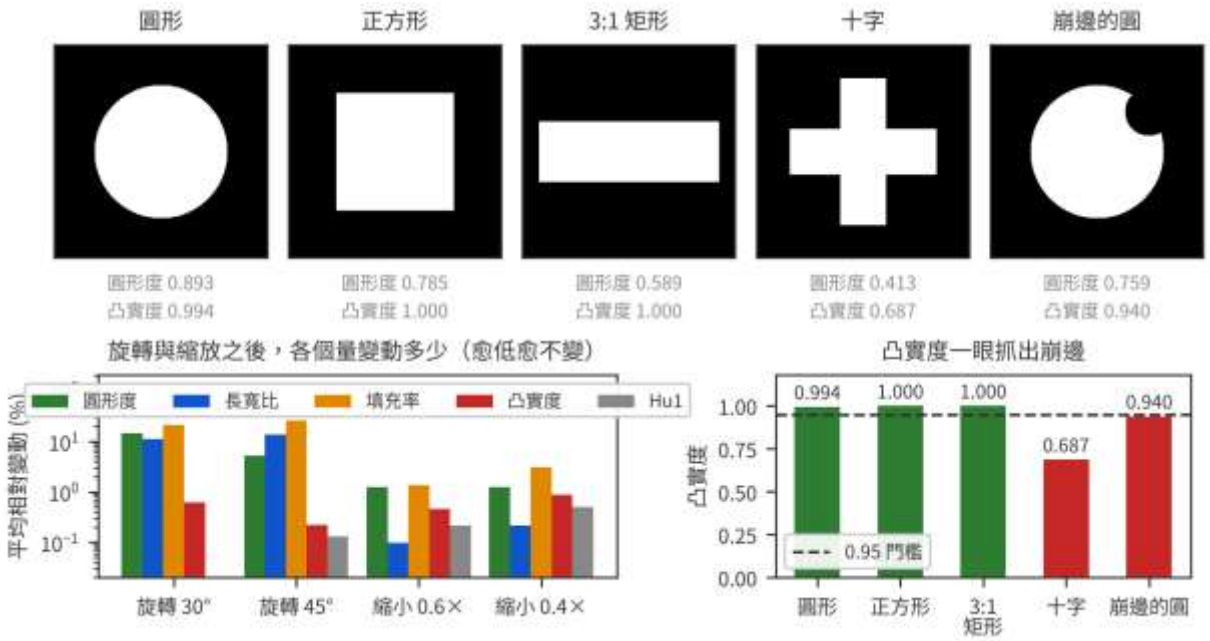
要更進一步、要能真正「辨認形狀」，就需要矩(moments)。空間矩定義為 m(p,q) = Σ x^p y^q I(x,y)，它把物件的質量分布對座標做加權統計。由此可得質心 (m10/m00, m01/m00);若把座標原點平移到質心再計算，得到的中心矩 μ(p,q) 自然就與物件的位置無關——取得平移不變性。再把中心矩除以適當次方的 μ00，得到正規化中心矩 η(p,q) = μ(p,q) / μ00^((p+q)/2+1)，尺度的影響也被約掉了——取得尺度不變性。

剩下的旋轉不變性最難。1962 年，胡明寬(Ming-Kuei Hu)在一篇經典論文中，以二階與三階的正規化中心矩巧妙組合出七個量，證明它們在平移、縮放與旋轉之下都保持不變——這就是著名的 Hu 不變矩。由於七個值的數量級差異極大(動輒跨越十

幾個數量級)，實務上一律先取 $-\text{sign}(h)\cdot\log_{10}|h|$ 壓縮動態範圍，再拿來比較或餵給分類器。

本書以一個不規則五邊形實測驗證了這些性質：把它平移之後，七個值完全相同;放大 1.8 倍或旋轉 37 度，在取對數的尺度下前四項的差異都不超過 0.02。還有一個有趣的性質值得記住：第七個不變矩在鏡像(左右翻轉)時會變號而絕對值不變，因此它可以用來區分一對互為鏡像的形狀——例如左旋與右旋的螺絲、左右手的手套。

Hu 不變矩也有清楚的侷限。高階矩對雜訊與輪廓的量化誤差相當敏感，對高度對稱的形狀還會出現數值接近零而不穩定的情況，所以實務上多半只採用前三到四項。若需要更強的形狀辨識能力，常見的升級路線有二：改用輪廓的傅立葉描述子(把輪廓視為一維週期訊號做 DFT，正是第 6 章的思想用在輪廓上)，或者乾脆把遮罩交給 CNN(第 15 章)去學。OpenCV 的 matchShapes 函式即以 Hu 矩為基礎，可直接回報兩個輪廓的形狀距離，值愈小愈相似。



形狀特徵的設計重點是「無因次」——用比值把單位約掉，量到的才是形狀本身，而不是拍攝距離。圓形度 $4\pi A/P^2$ 就是典型：面積與周長的平方都是長度平方，相除後單位消失。實測本圖五個形狀：圓形 0.893、正方形 0.785、3:1 矩形 0.589、十字 0.413，正方形與 3:1 矩形與理論值 0.785、0.589 完全相符，但圓形量到 0.893 而不是理論的 1.000——這不是公式錯了，是離散化的後果：圓在像素網格上是鋸齒狀的，沿著鋸齒走一圈量到的周長比真正的圓周長了約 6%，而周長在分母又被平方，誤差就被放大了。這是所有以周長為分母的指標共通的毛病，實務上若要精確，應先對輪廓做多邊形近似或平滑再量。左下是不變性的實測：把每個形狀旋轉與縮小之後，各個量變動多少。縮小 0.4 倍時，圓形度變動 1.2%、凸實度 0.9%、而 Hu1 變動 0.5%。要注意長寬比對旋轉完全不是不變量（旋轉 45 度時變動 14%），因為它用的是「與座標軸對齊」的外接矩形——這正是為什麼車牌偵測在斜拍時要先做透視校正。右下示範凸實度的用途：完整的凸形物件接近 1，而崩邊的圓只有 0.940，一刀就切出來了。瑕疵檢測常用它而不用圓形度，因為小缺口對周長的影響有限，對凸包面積的影響卻很直接。

圖 13-18　形狀特徵：哪些量真的不變，哪些只是看起來不變

13.5.5 三類手工特徵的選用指南

把本章至此介紹的特徵整理成一張表，可以看清它們各自的位置與適用場合。

表 13-2 三類手工特徵的比較

比較面向	關鍵點描述子	區域描述子	形狀特徵
代表方法	Harris、SIFT、ORB	HOG、LBP、Gabor	圓形度、凸實度、Hu 不變矩
取樣方式	稀疏:少數關鍵點	稠密:固定視窗全覆蓋	以單一物件為單位
輸出形式	每個點一個描述子	每個視窗一個向量	每個物件一組數值
主要不變性	尺度、旋轉	光照(LBP 最強)	平移、尺度、旋轉
需先分割嗎	不需要	不需要	需要:第 9、10 章的輸出

典型用途	拼接、定位、追蹤	行人偵測、紋理與瑕疵	零件量測與幾何篩選
下游接什麼	RANSAC 幾何求解	第 14 章的分類器	門檻篩選或分類器

三者並非互斥，實務上常常串成一條管線。以一條零件外觀檢測線為例：先用第 9 章門檻化與第 10 章型態學取出候選物件 → 用形狀特徵(面積、圓形度、凸實度)做幾何篩選，一秒之內淘汰明顯不合格與誤檢的區塊 → 對通過的物件逐格算 LBP 直方圖判斷表面是否有刮痕與鏽蝕 → 再加上第 8 章的顏色直方圖判斷色差 → 把這些向量串接起來交給第 14 章的 SVM 做最終分級。這正是第 14 章分級案例所用的特徵組合，也是台灣機械與電子業產線上最常見的做法：便宜、可解釋、樣本需求少。當產品換線時只要重新訓練分類器，不必重新標註上萬張照片。

最後要強調的是，這三類特徵並沒有被深度學習淘汰。在樣本稀少、算力受限、或必須向客戶解釋「為什麼判定不良」的場合，手工特徵往往比端到端的神經網路更合適——一個「凸實度低於 0.95 即判定崩邊」的規則，品保工程師看得懂、也願意簽名;而這恰好是 13.8 節要繼續討論的主題。

程式 13-4 HOG 與 LBP 特徵擷取

```
import cv2, numpy as np
from skimage.feature import local_binary_pattern

img = cv2.imread("person.jpg", cv2.IMREAD_GRAYSCALE)

# === HOG:方向梯度直方圖 ===
# 注意:cv2.HOGDescriptor 只存在於 OpenCV 4.x,
#      OpenCV 5 已移除其 Python 介面,直接呼叫會拋 AttributeError,
win = cv2.resize(img, (64, 128))      # 標準行人視窗
# hog = cv2.HOGDescriptor(    # 以下一律註解,改用後面的 skimage 版本
#         _winSize=(64, 128), _blockSize=(16, 16),
#         _blockStride=(8, 8), _cellSize=(8, 8),
#         _nbins=9)
# f_hog = hog.compute(win).ravel()
# print("HOG 維度:", f_hog.shape)  # (3780,)=105x4x9

# skimage 版本:結果完全等價,並可輸出方向場視覺化圖
from skimage.feature import hog as sk_hog
f2, vis = sk_hog(win, orientations=9,
                 pixels_per_cell=(8, 8),
                 cells_per_block=(2, 2),
                 block_norm="L2-Hys", visualize=True)
```

```
print("HOG 維度:", f2.shape)   # (3780,)=105x4x9,見 13.5.6 節

# === 內建的 HOG + SVM 行人偵測器:同樣只存在於 OpenCV 4.x ===
# det = cv2.HOGDescriptor()  # 5.x 需改裝 opencv-contrib-python
# det.setSVMDetector(
#     cv2.HOGDescriptor_getDefaultPeopleDetector())
# boxes, _ = det.detectMultiScale(img,       # 影像金字塔
#                                 winStride=(8, 8),
#                                 scale=1.05)
# print("偵測到的行人數:", len(boxes))

# === LBP:區域二值模式(紋理) ===
P, R = 8, 1                       # 8 鄰居、半徑 1
lbp = local_binary_pattern(img, P, R,
                           method="nri_uniform")
hist, _ = np.histogram(lbp, bins=59, range=(0, 59),
                       density=True)
print("均勻模式直方圖維度:", hist.shape)      # (59,)

# 旋轉不變版本:八鄰居時只剩 10 維
lbp_ri = local_binary_pattern(img, P, R,
                              method="uniform")
h_ri, _ = np.histogram(lbp_ri, bins=P + 2,
                       range=(0, P + 2), density=True)
print("旋轉不變 LBP 維度:", h_ri.shape)      # (10,)
```

程式 13-5 形狀特徵與 Hu 不變矩

```
import cv2, numpy as np, math

# 輸入:第 9 章分割 + 第 10 章型態學清理後的遮罩
mask = cv2.imread("part.png", cv2.IMREAD_GRAYSCALE)
_, bw = cv2.threshold(mask, 0, 255,
          cv2.THRESH_BINARY + cv2.THRESH_OTSU)
cnts, _ = cv2.findContours(bw, cv2.RETR_EXTERNAL,
                           cv2.CHAIN_APPROX_SIMPLE)

for c in cnts:
    A = cv2.contourArea(c)            # 面積
    if A < 200:                       # 濾掉雜點
        continue
    P = cv2.arcLength(c, True)        # 周長
    x, y, w, h = cv2.boundingRect(c)
    hull = cv2.convexHull(c)

    circ  = 4 * math.pi * A / (P * P)   # 圓形度 圓=1
    ratio = w / float(h)                # 長寬比
    ext   = A / float(w * h)            # 填充率
    sol   = A / cv2.contourArea(hull)   # 凸實度
    dia   = math.sqrt(4 * A / math.pi)  # 等效直徑
    print(f"面積{A:7.0f} 圓形度{circ:.3f} "
          f"長寬比{ratio:5.2f} 填充率{ext:.3f} "
          f"凸實度{sol:.3f} 等效直徑{dia:6.1f}")
```

```
    # --- Hu 不變矩:平移/尺度/旋轉不變 ---
    hu = cv2.HuMoments(cv2.moments(c)).ravel()
    # 取對數壓縮動態範圍;高階項不穩,實務多取前 3~4 項
    hu_log = [-np.sign(v) * math.log10(abs(v))
              if abs(v) > 1e-20 else 0.0 for v in hu]
    print("   log Hu:", np.round(hu_log, 3))

# 以 Hu 矩直接比較兩個輪廓的形狀(值愈小愈相似)
if len(cnts) >= 2:
    d = cv2.matchShapes(cnts[0], cnts[1],
                        cv2.CONTOURS_MATCH_I1, 0)
    print("形狀距離:", round(d, 4))
```

13.5.6 推導：HOG 的維度怎麼算，以及區塊為什麼要重疊

HOG 的 3,780 維常被當成一個要背的數字，其實它完全可以推出來，而推導的過程會順便解釋兩個設計決定。

先算區塊數。視窗 64×128，細胞 8×8，所以水平有 64÷8 = 8 行細胞、垂直有 128÷8 = 16 列細胞。區塊由 2×2 個細胞組成，並以「一個細胞」為步幅滑動，因此水平方向能放 8 − 2 + 1 = 7 個區塊，垂直方向 16 − 2 + 1 = 15 個，共 7 × 15 = 105 個區塊。每個區塊含 4 個細胞、每個細胞 9 個方向，所以總維度是 105 × 4 × 9 = 3,780。圖 13-15 用 skimage 實際算出來的也正是 3,780 維，兩者相符。

注意步幅是「一個細胞」而不是「一個區塊」，這表示區塊是重疊的——相鄰兩個區塊共用一半的細胞。若不重疊，區塊數會變成 4 × 8 = 32 個，維度只有 1,152 維。多出來的 2,628 維買到了什麼?答案是：同一個細胞會出現在最多四個區塊裡，被四組不同的鄰居各正規化一次。萬一某個區塊裡剛好有一道強光或一條高反差的邊，它會壓低該區塊裡其他細胞的相對權重;但同一個細胞在另外三個區塊裡仍有正常的表現，所以單一區塊的異常不會毀掉這個細胞的描述。用冗餘換強健，是很典型的工程交易。

再看正規化本身。圖 13-16 的實測必須先避開一個陷阱：不能用餘弦相似度來比較，因為餘弦對整體縮放免疫，而「對比度下降」正好就是整體縮放，用餘弦會完全看不見差別。改用相對 L2 距離之後結論就清楚了：把對比度壓到 0.2 倍時，完全不正規化的描述子整個縮小了 80%，而 L2-Hys 只變動 22.6%，差了 3.5 倍。道理很直接

——對比度下降讓所有梯度等比例變小，直方圖的形狀其實沒變、只是整體縮小，而正規化把每個區塊除以自己的長度，等比例的縮放就被除掉了。

L2-Hys 這個名字裡的 Hys 是 hysteresis（遲滯）：先做 L2 正規化，把任何超過 0.2 的分量截斷成 0.2，再做一次 L2 正規化。用意是壓制少數極強的梯度獨佔整個區塊的描述——例如衣服上的一塊高光或背景的一根電線桿，它們的梯度可能比人的輪廓強上好幾倍，若不截斷，整個區塊的描述就會被它主導，人的外形反而被稀釋掉。這個「把過強的證據打折」的想法，與第 5 章 CLAHE 的對比度限制如出一轍。

最後補一個與 SIFT 的對照，因為兩者的骨幹其實是同一個：梯度方向直方圖。三個關鍵差異是：取樣上 SIFT 稀疏（只在關鍵點）、HOG 稠密（固定網格全覆蓋）;方向上 SIFT 先量主方向再對齊、HOG 刻意不對齊;輸出上 SIFT 每個關鍵點一個 128 維向量、數量不固定，HOG 每個視窗一個固定長度的向量。其中「HOG 為什麼不做方向對齊」最值得想一想：因為它要辨識的是「站著的行人」，把每個區塊都轉正，等於丟掉了「這個人是直立的」這個最重要的線索。不變性不是愈多愈好——該對什麼不變，是由任務決定的。

13.6 應用：影像拼接、模板比對與物件定位

13.6.1 影像拼接(全景圖)

全景照片是特徵匹配最迷人的應用。原理：對兩張有重疊區域的照片各自擷取特徵 → 匹配並用 RANSAC 求出它們之間的透視轉換(第 4 章)→ 把一張影像依此轉換「拉」到與另一張對齊的位置 → 融合重疊區域(消除接縫)。手機相機的「全景模式」、Google 街景的環景圖，骨幹都是這條流程。它綜合了本章(特徵匹配)、第 4 章(透視轉換)、第 6 章(相位相關可做初始對齊)、第 11 章(多頻帶融合可消接縫)。



影像拼接把本章前面所有東西串成一條線，而且每一步都不能省。先用 SIFT 在兩張影像各取特徵、做最近鄰匹配，比值測試留下 220 組，RANSAC 再認可其中 209 組為內點，並由它們解出一個單應矩陣（第 4 章的透視轉換，8 個自由度，所以最少要 4 組點）。把右圖依這個矩陣扭到左圖的座標系，兩張就對齊了。但對齊不等於拼好——最後一步的「融合」常被初學者忽略。中間上方是直接覆蓋。本圖刻意讓右影像的曝光高一些（增益 1.22、偏移 14，模擬兩次拍攝的自動曝光差異），於是即使幾何完全正確，接縫處仍留下一條明顯的亮度斷層，實測這道階梯讓每欄平均亮度在一瞬之內跳了 35.3 階。中間下方做了最簡單的線性混合——在重疊區讓權重由 1 平滑降到 0。量的時候要小心：混合會把斷層攤開到整個重疊區，若只量接縫那幾欄，會得到「混合沒有用」的錯誤結論（本圖初稿就犯了這個錯），而且不能量局部梯度——重疊區本來就有高對比的影像內容，量到的會是內容而不是接縫。改成對整欄取平均再看變化率，影像內容被平均掉，剩下的就是曝光不一致造成的那道階梯。結果：尖峰從一欄跳 35.3 階降到 9.9 階，而且不再是一根尖刺，而是攤平在整個重疊區的一片小丘。實務上還有更好的做法：找一條沿著「兩張差異最小」的路徑來切（接縫雕刻），或在多個頻帶上分別混合（拉普拉斯金字塔融合，又是第 11 章的多解析度）。但觀念是同一個：讓過渡發生在人眼不容易察覺的地方。

圖 13-19　影像拼接：對齊之後，還有融合這一步

13.6.2 模板比對與其侷限

模板比對(Template Matching)是最樸素的「找物件」方法：拿一個小範本(template)在大影像上逐位置滑動，計算每個位置的相似度，相似度最高處就是物件所在。OpenCV 的 matchTemplate 提供此功能。它簡單直接，但侷限明顯：對尺度與旋轉極度敏感——範本一旦與目標的大小或角度不符就會失敗。這恰恰反襯出特徵匹配(SIFT/ORB)的價值：後者的尺度與旋轉不變性，正是為了克服模板比對的死穴而生。何時用哪個?目標大小角度固定(如生產線上位置固定的工件)用模板比對即可;目標會變大變小變角度，就需要特徵匹配。

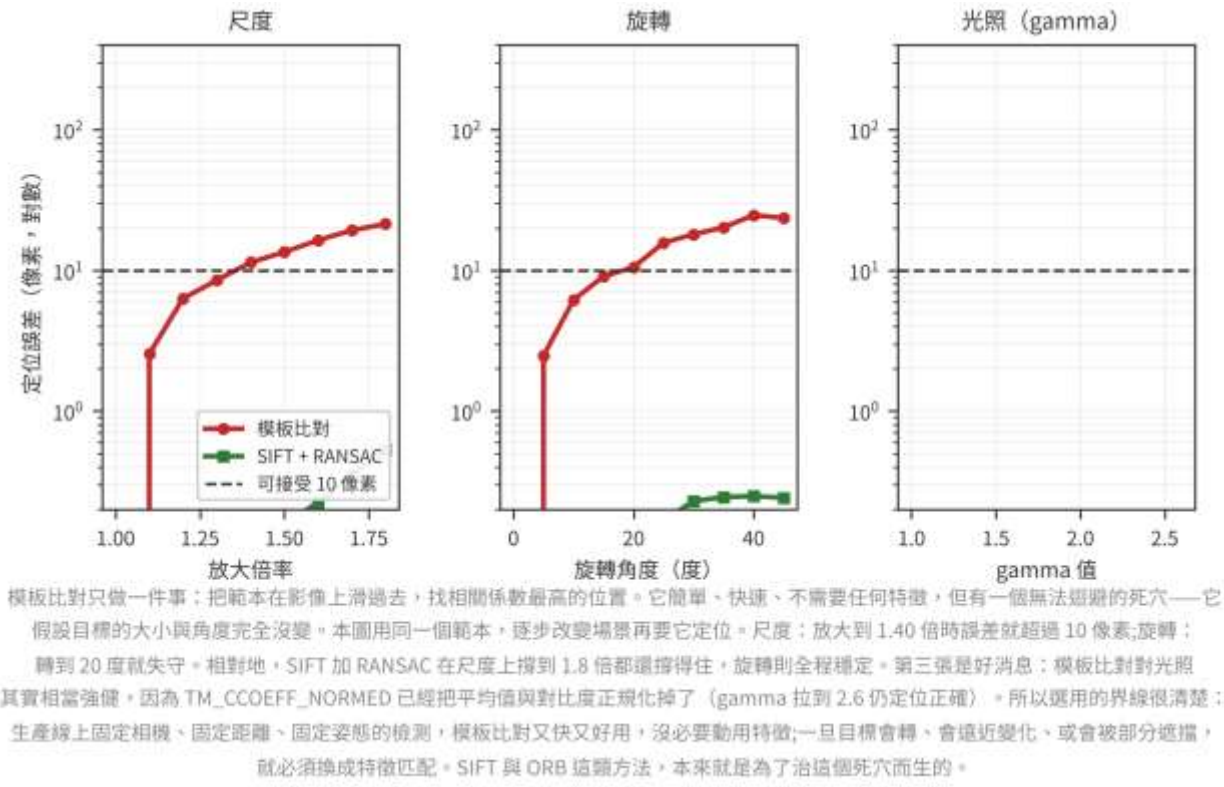


圖 13-20　模板比對的死穴：尺度與旋轉，以及它其實不怕的東西

13.6.3 物件定位

程式 13-3 已示範了完整的物件定位：給定一張「目標物件」的參考圖與一張「場景」圖，透過特徵匹配加 RANSAC，不僅能判斷場景中「有沒有」該物件，還能精確框出它「在哪裡、以什麼角度出現」——即使物件被部分遮擋(SIFT 論文指出，少至三個正確匹配的特徵就足以定位物件)。這是深度學習物件偵測(第 16 章)普及之前，工業界物件辨識與定位的主力方法，至今在樣本稀少、不便訓練深度模型的場合仍有一席之地。

13.7 案例深探：車牌偵測

車牌偵測是一個絕佳的綜合案例，它把本書多章的技術串成一條完整的實用管線，也是您在課程中反覆演練的經典題目。完整的車牌辨識(License Plate Recognition,LPR)分為「偵測定位」與「字元辨識」兩大階段，本節聚焦前者(偵測)，字元辨識則銜接第 10 章(字元分割)與第 15 章(字元分類)。

13.7.1 傳統車牌偵測管線

傳統方法充分利用車牌的視覺特性——矩形、高對比、字元密集(高頻紋理豐富)。典型流程：

20. 前處理：轉灰階、以第 5 章的方法增強對比、去雜訊。
21. 邊緣與紋理分析：車牌區域因字元密集而有大量垂直邊緣。用第 9 章的 Sobel 求垂直梯度，車牌區域會呈現密集的邊緣響應，凸顯出候選區域。
22. 型態學處理：用第 10 章的閉合運算把密集的邊緣連成一整塊矩形區域，再用斷開去除雜點。
23. 候選區域篩選：用第 10 章的連通元件標記找出候選區塊，依車牌的長寬比(約 3:1 到 4:1)、面積、填充率等幾何條件過濾掉不像車牌的區域。
24. 透視校正：若車牌是斜拍的，用第 4 章的透視轉換把它「拉正」成正面矩形，方便後續字元分割。

這條管線是第 4、5、9、10 章的完美綜合，展示了傳統影像處理如何以「一連串簡單步驟的巧妙串接」解決真實問題——這正是叫獸開講中 Canny、Viola-Jones 都體現過的工程智慧。

13.7.2 特徵匹配在車牌辨識中的角色

特徵匹配(本章主題)在車牌辨識中也有用武之地：例如以特徵匹配比對車牌上的地區標誌或特殊符號、或在多影格影片中追蹤同一張車牌(結合第 12 章)。而在現代系統中，整個偵測階段愈來愈多改用第 16 章的 YOLO——直接訓練一個「車牌偵測器」，一步到位框出車牌，對傾斜、模糊、光線變化的強健度遠勝傳統管線。這再次呼應全書的主軸：傳統方法(可解釋、不需訓練、樣本少也能用)與深度學習(強健、需資料與算力)各擅勝場，成熟的工程師懂得依場景選擇，甚至混用。

程式 13-6 傳統車牌偵測管線

```
import cv2, numpy as np
```

```
img = cv2.imread("car.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. 前處理(第 5 章)
gray = cv2.bilateralFilter(gray, 11, 17, 17)   # 保邊去雜訊

# 2. 垂直邊緣:車牌字元密集,垂直邊緣豐富(第 9 章)
edges = cv2.Sobel(gray, cv2.CV_8U, 1, 0, ksize=3)
_, edges = cv2.threshold(edges, 0, 255,
           cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 3. 型態學:把密集邊緣連成矩形塊(第 10 章)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (17, 5))
closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel)

# 4. 連通元件 + 幾何條件篩選候選車牌(第 10 章)
cnts, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL,
                           cv2.CHAIN_APPROX_SIMPLE)
for c in cnts:
    x, y, w, h = cv2.boundingRect(c)
    ratio = w / float(h)
    if 2.5 < ratio < 5.0 and w > 80:      # 車牌長寬比約 3~4:1
        cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
        plate = gray[y:y+h, x:x+w]        # 裁出車牌供字元分割(第 10 章)

cv2.imwrite("plate_detected.png", img)
```

13.8 從手工特徵到學習特徵

本章介紹的 Harris、SIFT、ORB 與 HOG、LBP、形狀特徵，都屬於手工設計特徵(hand-crafted features)——它們是人類專家憑藉對影像的深刻理解，精心設計出來的偵測與描述規則。在深度學習興起前的二十年，手工特徵是電腦視覺的絕對主流，SIFT 更是其中的皇冠。

2012 年後，深度學習帶來了典範轉移(這正是第 15 章的主題)：卷積神經網路不再依賴人類設計特徵，而是從海量資料中自動學習特徵。但這裡有一個深刻且美麗的連結——第 6.6.7 節已經預告過：當我們把訓練好的 CNN 第一層卷積核視覺化，會發現它們自發演化成了邊緣偵測器與方向性帶通濾波器，與人工設計的 Gabor、SIFT 的梯度方向分析驚人地相似!換言之，深度學習並沒有推翻手工特徵的智慧，而是「重新發明」並「自動化」了它——網路透過梯度下降，自己學會了人類專家花了數十年才設計出的東西，並且能組合出更複雜、更抽象的高層特徵。

因此，學習本章絕非學習「過時的技術」。理由有三：其一，理解手工特徵，才能真正理解 CNN 學到了什麼、為什麼有效——手工特徵是深度特徵的「白箱版本」。其二，在樣本稀少、算力受限、需要可解釋性的場合(許多工業與機器人場景)，手工特徵至今仍是最務實的選擇。其三，現代許多系統是混合式的：用深度學習做高層理解、用手工特徵(如 ORB)做即時的幾何對齊與定位(例如許多 SLAM 系統的核心仍是 ORB 特徵)。手工特徵與學習特徵不是取代關係，而是各據所長、相互補充——這也是本章作為「傳統視覺」與「現代視覺」橋梁的意義所在。

13.9 本章與全書的觀念連結總表

相關章節	共用的觀念	特徵擷取中的角色
第 4 章 幾何轉換	透視轉換、單應性	RANSAC 求轉換做物件定位與影像拼接
第 5 章 空間域強化	高斯模糊、直方圖、正規化	SIFT 尺度空間、方向直方圖、描述子正規化
第 6 章 頻率域	帶通、Gabor、DoG	SIFT 的 DoG 即帶通;Gabor 為紋理特徵
第 8 章 色彩處理	顏色直方圖、紋理	LBP 判斷表面瑕疵;顏色直方圖為區域描述子
第 9 章 影像分割	梯度、Sobel、LoG	角點與 SIFT 建立於梯度;DoG 近似 LoG
第 10 章 二值影像	型態學、連通元件	車牌偵測的候選區域生成與篩選
第 11 章 小波	多解析度分析	SIFT 尺度金字塔與小波多尺度同源
第 12 章 視訊處理	光流、追蹤、視訊穩定	Shi-Tomasi 供光流追蹤;特徵匹配做穩定
第 14 章 機器	SVM、特徵工程	HOG/LBP/形狀特徵即傳統機器學習的輸

學習		入
第 15、16 章 深度學習	CNN 卷積核、物件偵測	CNN 首層重現手工特徵;YOLO 取代傳統偵測
第 17 章 邊緣部署	即時性、SLAM	ORB 輕量，為機器人 SLAM 的核心特徵

表 13-3 第 13 章與全書其他章節的觀念連結

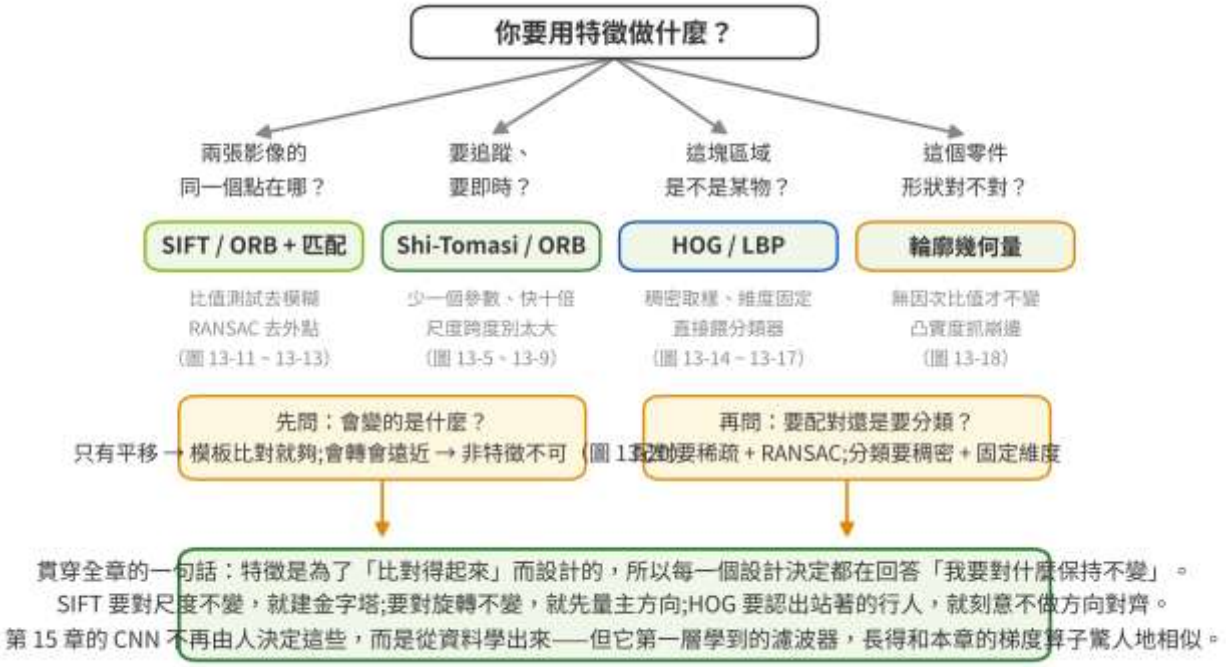


圖 13-21　第 13 章的決策地圖

程式實作 Lab 13

Lab 13-1 角點偵測與特徵比較

目標：建立對特徵的直覺。(a)對同一張影像分別套用 Harris、Shi-Tomasi、SIFT、ORB，比較各自偵測到的關鍵點分布——它們都偏好落在什麼樣的區域?(b)把影像放大兩倍、旋轉 45 度、調暗，重新偵測，量化比較哪種方法的「重複性」最好(同一特徵在變換後仍被偵測到的比例);(c)驗證角點偵測不具尺度不變性：放大影像後，Harris 角點是否消失?SIFT 是否仍找得到?(d)討論：為什麼平坦區與邊緣上偵測不到角點?這與第 12 章孔徑問題有何關聯?

Lab 13-2 物件定位器

目標：實作程式 13-3 的完整物件定位。(a)拍一張小物件(如書本封面、商標)作為「目標」，再拍幾張把它放進雜亂場景(不同角度、部分遮擋)的照片作為「場景」;(b)用 SIFT/ORB 加 BF 匹配加比值測試加 RANSAC，在場景中框出目標的位置與方向;(c)測試極限：目標旋轉多少、遮擋多少、縮放多少時開始失敗?(d)比較 SIFT 與 ORB 的定位成功率與速度(量測 FPS);(e)討論：比值測試的門檻(0.7、0.75、0.8)如何影響匹配的數量與品質?RANSAC 剔除了多少外點?

Lab 13-3 全景圖拼接

目標：做出自己的全景照片。(a)手持相機水平平移，拍攝三到五張有重疊(建議重疊三成以上)的照片;(b)相鄰兩張擷取特徵、匹配、RANSAC 求透視轉換，把後一張拉到與前一張對齊，逐步拼接;(c)處理接縫：比較「直接覆蓋」與「重疊區漸變融合」的視覺差異(進階可用第 11 章的多頻帶融合);(d)直接用 OpenCV 的 cv2.Stitcher 對照你的手工結果;(e)討論：什麼情況下拼接會失敗(重疊太少、場景無紋理、有移動物件)?

Lab 13-4 HOG 與 LBP 的手感實驗

目標：親手驗證兩種區域描述子的性質。(a)對一張含行人的照片，用程式 13-4 輸出 HOG 的方向場視覺化圖，觀察人體輪廓是否真的在方向場中浮現;(b)用內建的 HOG+SVM 偵測器跑同一張照片，再把照片旋轉 30 度後重跑，說明偵測率為何明顯下降——這驗證了 HOG 不具旋轉不變性;(c)對一張紋理照片(布料、木紋或金屬表面)計算 59 維均勻模式 LBP 直方圖;(d)把同一張照片做 Gamma 校正與線性調亮後重算 LBP，比較兩個直方圖的差異，驗證 LBP 對單調灰階變換的不變性;(e)取兩種不同材質各五張照片，以直方圖距離(如卡方距離)判斷「同材質是否真的比較近」。

Lab 13-5 零件形狀量測與幾何篩選

目標：建立一條最便宜的瑕疵篩選管線。(a)拍攝一批同款零件(或以硬幣、墊片、螺帽代替)，刻意混入數個有缺角、崩邊或變形的樣本;(b)以第 9 章門檻化加第 10 章型

態學取出每個物件的輪廓;(c)用程式 13-5 計算面積、圓形度、長寬比、填充率、凸實度與等效直徑，整理成一張表;(d)觀察哪一個指標對「缺角」最敏感(提示：先看凸實度)，訂出一組門檻值，計算你的篩選規則在這批樣本上的誤判率;(e)對同一個零件分別拍攝正放、放大與旋轉 45 度三張照片，計算 Hu 不變矩並比較前四項，驗證其不變性;(f)以 cv2.matchShapes 比較良品與不良品的形狀距離，討論這個單一數值能否取代你在 (d) 訂出的多條規則。

Lab 13-6 車牌偵測管線

目標：實作 13.7 節的傳統車牌偵測。(a)以程式 13-6 為基礎，對車輛照片完成「前處理→垂直邊緣→型態學→連通元件→幾何篩選」的完整管線，框出車牌;(b)調校型態學結構元素的尺寸與長寬比門檻，提高偵測率、降低誤報;(c)對斜拍的車牌，用第 4 章透視轉換拉正;(d)串接第 10 章：對拉正後的車牌做二值化與連通元件，分割出各個字元;(e)討論：傳統管線在什麼情況下失敗(強光、髒污、傾斜過大)?對照第 16 章，YOLO 車牌偵測會如何改善這些問題?各自的成本與適用場景為何?

本章重點整理

- 特徵是影像中值得注意、易辨認、可重複找到的局部結構;好特徵須具重複性、獨特性、不變性(尺度/旋轉/光照)。角點在兩方向都有變化，能唯一定位，是理想特徵(呼應第 12 章孔徑問題)。
- Harris 角點以視窗滑動的灰階變化與結構張量特徵值判斷：兩者都大即角點;Shi-Tomasi 取較小特徵值作強度，更適合追蹤，為 goodFeaturesToTrack 的預設;兩者皆不具尺度不變性。
- SIFT 四步驟(尺度空間 DoG 極值、關鍵點定位、方向指派、128 維描述子)達成尺度與旋轉不變，是第 5、6、9、11 章工具的集大成;ORB(FAST+BRIEF)免費、快數十倍、二進位描述子用漢明距離比對，適合即時與邊緣裝置。

- 匹配：BF 準確、FLANN 快速近似;Lowe 比值測試(最近/次近距離比)過濾模糊匹配;RANSAC 以「投票」思想反覆抽樣建模、剔除外點，精神與霍夫轉換一致。
- 應用：影像拼接(特徵匹配+透視轉換+融合)、模板比對(簡單但不具尺度旋轉不變性)、物件定位(匹配+RANSAC，可抗遮擋);車牌偵測綜合第 4、5、9、10 章，是傳統管線的典範。
- Harris/SIFT/ORB 為手工特徵;CNN 首層自動學出的濾波器與之高度相似，深度學習是「重新發明並自動化」了手工特徵。兩者互補：手工特徵可解釋、樣本少可用、即時(SLAM 用 ORB)，學習特徵強健、能學高層語意。
- HOG 以「無號梯度方向直方圖 + 區塊 L2-Hys 正規化」描述外形:8×8 細胞、9 個方向、2×2 細胞成區塊且重疊滑動;64×128 視窗得 105×4×9 = 3,780 維。與 SIFT 同為梯度方向直方圖，但稠密取樣且刻意不對齊方向，故不具旋轉不變性，尺度靠影像金字塔加滑動視窗解決;HOG+SVM 是 YOLO 之前行人偵測的標準答案。
- LBP 以 3×3 鄰域「比中心亮記 1」串成八位元碼，其直方圖即紋理描述。因只比大小不比數值，對單調遞增的灰階變換完全不變，光照強健性極佳;取跳變次數不超過兩次的 58 個均勻模式加一個併合區間，可由 256 維降到 59 維而幾乎不損資訊;另有旋轉不變與多尺度 LBP(P,R) 延伸。運算極廉，是表面瑕疵檢測的性價比首選。
- 形狀特徵以第 9、10 章切出的輪廓為輸入，用無因次比值取得尺度不變性:圓形度 $4\pi A/P^2$(圓 1.000、正方形 0.785、3:1 矩形 0.589)、長寬比、填充率、凸實度(對缺角最敏感)、等效直徑。矩的三級遞進為:中心矩得平移不變 → 正規化中心矩得尺度不變 → Hu 的七個組合得旋轉不變;實務取對數並只用前三到四項，第七項在鏡像時變號可區分左右形狀。

習題

1. 什麼是影像的「特徵」?為什麼用少數特徵比對比逐像素比對更好?請用一個生活比喻說明。

   提示與參考解答：特徵是影像中值得注意、易辨認、可重複找到的局部結構(如角點、獨特斑塊)。用少數特徵比對高效且強健，避免比對成千上萬像素。生活比喻：描述一個人不會逐一報告每個毛孔，而是說「濃眉、鷹勾鼻、左臉有痣」——用少數獨特標記就能精準辨認。

2. 好的特徵應具備哪三個條件?各舉一個「不符合」該條件的反例。

   提示與參考解答：重複性(不同條件下都能再次偵測到;反例：只在特定光線下出現的反光點)、獨特性(與眾不同易區分;反例：平坦牆面上的點，每點都一樣)、不變性(對尺度/旋轉/光照穩定;反例：Harris 角點放大後消失，不具尺度不變性)。

3. 為什麼角點是理想的特徵，而平坦區與邊緣不是?這與第 12 章的孔徑問題有何關聯?

   提示與參考解答：角點在兩個方向都有明顯灰階變化，能被唯一定位(重複+獨特)。平坦區任何方向移動都無變化，無法定位;邊緣沿邊緣方向移動無變化，只能定出垂直方向(一維模糊)。這正是第 12 章孔徑問題的體現——局部只能看出垂直於邊緣的運動/位置，唯有角點無此歧義。

4. 說明 Harris 角點的核心直覺，以及結構張量兩個特徵值的三種組合分別代表什麼。

   提示與參考解答：直覺：小視窗在影像上滑動，觀察灰階變化——平坦區任何方向都不變、邊緣僅垂直方向變、角點任何方向都劇烈變。結構張量兩特徵值：都小=平坦區，一大一小=邊緣，都大=角點。Harris 用不需實際算特徵值的響應函數 R 高效判斷。

5. Shi-Tomasi 相對 Harris 做了什麼改良?為什麼它更適合用於第 12 章的光流追蹤?

   提示與參考解答：Shi-Tomasi 直接取兩個特徵值中較小者作為角點強度，只要較小者夠大就保證兩方向都有足夠變化，判準更穩定。因此在追蹤場合表現更好，OpenCV 的 goodFeaturesToTrack 預設採用它，論文名〈Good Features to Track〉即點明與光流的血緣——這些角點是 Lucas-Kanade 稀疏光流理想的追蹤

點。

6. SIFT 如何達成尺度不變性與旋轉不變性?請分別說明對應的步驟。

   提示與參考解答：尺度不變性：建構高斯尺度金字塔(多解析度)，在「空間+尺度」三維空間找 DoG 極值——物體無論多大都能在對應尺度層被找到。旋轉不變性：分析關鍵點周圍梯度方向分布，指派主方向為基準，之後所有描述相對此基準計算——物體轉多少度，基準跟著轉，描述保持一致。

7. SIFT 的四個步驟分別用到本書前面的哪些概念?這說明了什麼學習上的道理?

   提示與參考解答：高斯模糊(第 5 章)、多尺度金字塔(第 11 章)、DoG 帶通/近似 LoG(第 6、9 章)、梯度(第 9 章)、方向直方圖(第 5 章)、描述子正規化抗光照(第 5 章)。說明 SIFT 是前面各章工具的集大成;循序學完基礎章節，是理解進階演算法的積木——基礎不牢，進階難懂。

8. ORB 為什麼比 SIFT 快?它由哪兩個技術融合而成?二進位描述子如何加速比對?

   提示與參考解答：ORB 融合 FAST 角點偵測器(極快)加方向資訊，與 BRIEF 二進位描述子(一連串「這點比那點亮嗎」的是非比較，產生 0/1 字串)加旋轉校正。二進位描述子比對時用漢明距離(算兩串位元有幾位不同)，可用 CPU 指令極速完成，遠快於 SIFT 的 128 維浮點歐氏距離運算。

9. 承叫獸開講：為什麼在 2020 年之前，免費的 ORB 儘管精度稍遜 SIFT 卻被廣泛採用?這對「技術選型」有何啟示?

   提示與參考解答：因為 SIFT 受專利保護(US 6,711,293,2020 年到期)，商業使用需授權;ORB 完全免費可自由商用。啟示：技術選型不只看效能，還要看授權、成本、生態系——工程決策不在真空中做，一個「夠好且無授權負擔」的方案常勝過「最強但受限」的方案(呼應第 11 章 JPEG vs. JPEG 2000)。

10. 說明 Lowe 比值測試的原理。為什麼「最近距離明顯小於次近距離」代表匹配可信?

    提示與參考解答：對每個特徵找出另一影像中最近與次近兩個匹配，若最近距離明顯小於次近(如小於 0.75 倍)則匹配可信，否則捨棄。原理：若一個特徵在對方影像中有唯一明確的對應，最近的會遠比次近的近;若最近與次近相當，代表有多個相似候選、難以區分，很可能匹配錯誤，應捨棄。

11. RANSAC 如何從充滿錯誤的匹配中找出正確的幾何模型?它與第 9 章的哪個演算法精神相同?

    提示與參考解答：RANSAC 反覆隨機抽取少數匹配、假設正確、據此算出幾何模型(如透視轉換)，再統計有多少其他匹配符合此模型;重複多次，得票最高(支持者最多)的模型勝出，不符者為外點剔除。與第 9 章霍夫轉換精神相同——都是「投票」：用多數證據對抗少數雜訊，是穩健估計的核心。

12. 模板比對的最大侷限是什麼?特徵匹配如何克服?各自適合什麼場合?

    提示與參考解答：模板比對對尺度與旋轉極度敏感，範本一旦與目標大小/角度不符就失敗。特徵匹配(SIFT/ORB)具尺度與旋轉不變性，正是為克服此死穴而生。場合：目標大小角度固定(如產線上位置固定的工件)用模板比對即可;目標會變大小變角度，需用特徵匹配。

13. (綜合題)請設計一條傳統車牌偵測管線，列出每一步用到本書哪一章的技術，並說明相對於第 16 章 YOLO 方案的優缺點。

    提示與參考解答：管線：前處理(第 5 章：灰階、去雜訊、增強)→ 垂直邊緣分析(第 9 章 Sobel，車牌字元密集邊緣多)→ 型態學閉合連成矩形塊、斷開去雜(第 10 章)→ 連通元件標記加長寬比/面積/填充率篩選(第 10 章)→ 透視校正拉正(第 4 章)→ 字元分割(第 10 章)。相對 YOLO：傳統管線可解釋、不需訓練資料、樣本少也能用、算力低;但對強光、髒污、大傾斜較不強健。YOLO 強健但需大量標註與 GPU。依場景與資源選擇或混用。

14. HOG 的四個計算步驟分別是什麼?為什麼第二步要用「直方圖」而不是直接保留每個像素的梯度方向?

    提示與參考解答：四步為(1)以 Sobel 求梯度大小與無號方向(0~180 度)、(2)在 8×8 細胞內以梯度大小為權重投票到 9 個方向區間、(3)把 2×2 細胞合為區塊並做 L2-Hys 正規化(區塊重疊滑動)、(4)串接成向量。用直方圖的理由是容忍度：直方圖只記錄「方向的分布」而不記錄「哪個像素是哪個方向」，因此細胞內的小幅位移、姿態與體型差異都不會改變統計結果，這與第 5 章用直方圖描述整體亮度分布的思想完全一致。

15. 一個 64×128 的行人視窗，在細胞 8×8、區塊 2×2 細胞、步幅 8 像素、9 個方向的設定下，HOG 描述子有幾維?請寫出計算過程，並說明這個數字反映了手工特徵的什麼代價。

    提示與參考解答：水平 64÷8−2+1 = 7、垂直 128÷8−2+1 = 15，共 105 個區塊;105×(2×2)×9 = 3,780 維。代價有二：其一是維度膨脹——描述一個小視窗就要近四千個數字，計算與儲存都不便宜;其二是這套規則(細胞大小、方向數、正規化方式)全由人訂，換一個任務往往要重新試誤調參，而第 15 章的 CNN 正是把這份工作交給資料與最佳化演算法。

16. 為什麼 LBP 對光照變化特別強健?什麼是均勻模式?為什麼取 58 個均勻碼加一個併合區間就能從 256 維降到 59 維而幾乎不損資訊?

    提示與參考解答：LBP 只問「鄰居是否比中心亮」，只比較大小、不使用灰階差值，因此任何單調遞增的灰階變換(Gamma、線性調亮、整體降對比)都不會翻轉比較結果，碼完全不變。均勻模式指八位元環上 0 與 1 的跳變次數不超過兩次的碼，它們對應平坦面、邊緣、線端與角點等真正有意義的局部結構，八鄰居時共 58 個(佔 256 碼的 22.7%)，卻覆蓋自然紋理約九成的像素;跳變頻繁的非均勻碼多半來自雜訊，彼此之間的區別意義不大，故全部併為一個區間即可。

17. 圓形度定義為 $4\pi A/P^2$。請說明它為什麼是無因次量，計算正方形與 3:1 矩形的圓形度，並說明在零件檢測中「凸實度」為何常比圓形度更適合用來發現崩邊。

    提示與參考解答：面積的單位是長度平方、周長的平方也是長度平方，相除後單位互相約掉，故為無因次量，對影像縮放與拍攝距離不敏感。正方形邊長 a:$4\pi\cdot a^2/(4a)^2 = \pi/4 \approx 0.785$;3:1 矩形(300×100):$4\pi\cdot 30000/(2\times 400)^2 \approx 0.589$。圓形度衡量的是整體輪廓的緊緻程度，一個小缺角對面積與周長的影響都很小，訊號容易被淹沒;凸實度 A/A 凸包 則是拿物件與「補滿凹處後的自己」相比，缺角、崩邊與凹口會直接造成凸包面積增加而使比值下降，因此靈敏度高得多。

18. Hu 不變矩如何逐步達成平移、尺度與旋轉不變?第七個不變矩有什麼特殊性質?為什麼實務上通常只使用前三到四項?

    提示與參考解答：三級遞進——把原點平移到質心得到中心矩 $\mu(p,q)$，位置的影響

消失(平移不變);再除以適當次方的 μ00 得到正規化中心矩 η(p,q) = μ(p,q)/μ00^((p+q)/2+1)，尺度的影響消失(尺度不變);Hu(1962)再以二階與三階的 η 組合出七個特定式子，使其在旋轉下亦不變。第七項在鏡像(左右翻轉)時絕對值不變但正負號翻轉，因此可用來區分互為鏡像的形狀(如左旋與右旋螺絲)。只用前三到四項的原因是高階矩由高階統計量構成，對雜訊與輪廓的量化誤差極為敏感，對高度對稱的形狀還會逼近零而數值不穩。

19. (綜合題)請比較 HOG 與 SIFT：兩者的共同骨幹是什麼?三個關鍵差異是什麼?為什麼 HOG 刻意不做方向對齊?並說明第 16 章的 YOLO 保留了這條管線的哪些零件、取代了哪些。

    提示與參考解答：共同骨幹都是「梯度方向直方圖」。差異：(1)取樣——SIFT 稀疏(只在關鍵點)、HOG 稠密(固定視窗全覆蓋);(2)方向對齊——SIFT 對齊關鍵點主方向以取得旋轉不變性、HOG 不對齊;(3)用途——SIFT 用於跨影像配對(對應問題)、HOG 用於視窗分類(辨識問題)。HOG 不對齊是刻意的設計：行人在影像中幾乎總是直立，「肩線偏水平、軀幹兩側偏垂直」本身就是最有價值的判別線索，對齊會把它洗掉。YOLO 保留了「影像金字塔/多尺度、密集特徵、非極大值抑制」這些零件，取代掉的是「人工設計的 HOG 描述子 + 線性 SVM + 滑動視窗」，改為卷積自動學出的特徵加上一次前向傳遞直接回歸所有框。

20. 「好特徵要有重複性」要怎麼量?請設計一個實驗，說明為什麼評估不變性時要避開旋轉 90 度這種角度，以及為什麼 SIFT 在旋轉、雜訊、光照三項測試中都不是最好的，卻仍被稱為劃時代的方法。

    提示與參考解答：做法是把同一張影像以已知的轉換處理一次，再問「原圖找到的關鍵點，經過同樣的轉換之後，有多少比例能在新影像中找到 3 像素以內的夥伴」，這個比例就是重複性。因為轉換已知，正確答案完全可控。要避開 90 度是因為它只是把像素重新排列、沒有重新取樣，不產生插值誤差，重複性會回升到接近 100%，是不具代表性的特例。圖 13-2 實測：旋轉 40 度時 Harris 74%、Shi-Tomasi 74%、SIFT 66%。SIFT 不佔優並不矛盾——旋轉與光照的不變性，Harris 這類基於梯度的偵測器本來就具備（結構張量隨影像一起旋轉）。SIFT 真正的本事在尺度，而那是這三項測試沒有涵蓋的（見下一題）。

21. 要證明「SIFT 具尺度不變性而 Harris 沒有」，該用放大還是縮小來測?該量偵測器還是量匹配?請說明兩個選擇的理由，並寫出你預期的結果。

    提示與參考解答：兩個選擇都有陷阱。第一，要用縮小而不是放大：物體變遠時資訊是真的被聚合掉的，而放大只是內插，並沒有製造出新的尺度內容，測不到重點。第二，要量匹配而不是量偵測。若只量「偵測器還找不找得到同一個位置」，三種方法在縮放下的表現其實差不多，因為角點縮小之後仍然是角點、位置大致還在。真正的差別在描述子——它必須知道「要用多大的鄰域來描述這個點」，而固定大小的鄰域在縮放後看到的根本是不同的內容。圖 13-6 實測匹配正確率：縮到 0.33 倍時，Harris 配固定大小的區塊描述子只剩 15%，ORB 57%，而 SIFT 仍有 82%。所以「尺度不變」不變的不只是偵測，更是描述。

22. SIFT 用「高斯差」(DoG) 而不是直接算 LoG，為什麼這樣做是合理的?請從熱傳導方程式說明這個近似的來源，並說明它為 SIFT 省下了什麼。

    提示與參考解答：兩個寬度略不同的高斯相減，會得到一個中間正、周圍負的帶通核，也就是第 9 章的墨西哥帽（LoG）。嚴格的來源是熱傳導方程式 $\partial G/\partial\sigma = \sigma\cdot\nabla^2 G$，把左邊的微分換成差分即得 $G(k\sigma) - G(\sigma) \approx (k-1)\sigma^2\cdot\nabla^2 G$。圖 13-7 把兩邊實際算出來比對，相關係數高達 0.9941。省下的是運算：LoG 要做二階微分、成本高，而高斯模糊無論如何都要做（建尺度金字塔本來就要），相減則幾乎不花錢。SIFT 等於用一個「順手」的運算換到了昂貴的帶通濾波器，這是演算法設計上非常漂亮的一步。

23. 比值測試為什麼看「最近 ÷ 次近」而不是看「最近的距離夠不夠小」?若要為自己的資料挑門檻，該怎麼做?過濾前後的正確率大約會差多少?

    提示與參考解答：因為「距離夠小」沒有絕對標準——不同影像、不同紋理，描述子距離的尺度完全不同。而「最近比次近近多少」是相對量，自動適應了這個尺度。背後的道理：若一個特徵在對方影像中真的有唯一對應，最近鄰應該明顯勝過第二名;若第一名與第二名難分軒輊，代表這個特徵長得很普通（重複紋理、磚牆、樹葉），配哪一個都不可靠。挑門檻的做法是用已知的幾何轉換造一對影像，這樣每個匹配是對是錯都查得出來，再掃描門檻看精確率與召回率。圖 13-11 實測：原始 1,019 個匹配只有 61.1% 正確;門檻 0.75 留下 609 個，正確率升到 98.2%、召回率 96.0%;最佳門檻落在 0.76，與 Lowe 論文建議的 0.75 幾乎一致。

24. RANSAC 要抽幾次才夠?請推導 N = log(1−p) / log(1−w^s)，並用它說明三件事：(a) 為什麼比值測試必須先做;(b) 為什麼「能用仿射就別用單應」;(c) 當 RANSAC 常常失敗時，該先檢查什麼。

    提示與參考解答：設內點比例為 w、建模最少需 s 組匹配。一次抽樣全中的機率是 w^s，連抽 N 次都沒全中的機率是 (1−w^s)^N，要讓成功機率達 p，需 1 − (1−w^s)^N ≥ p，取對數整理即得該式。(a) 圖 13-11 實測比值測試把正確率從 61.1% 拉到 98.2%，代入公式（s=4、p=0.99），所需抽樣從 29 次降到 2 次——兩者是一前一後的搭配，不是二選一。(b) N 對 s 極度敏感：內點 50% 時 s=2 只要 16 次、s=4 要 71 次、s=8 要 1,177 次。自由度低的模型不只快，也不容易被外點誘導出「扭曲但碰巧符合」的解。(c) 該先檢查內點比例，而不是一味調高迭代次數——N 對 w 是指數式敏感（內點 20% 要 2,876 次、10% 要 46,049 次），把 w 拉高永遠比把 N 調高划算。詳見 13.4.4 節。

25. (綜合題)請比較 SIFT 與 HOG：兩者的共同骨幹是什麼?三個關鍵差異是什麼?為什麼 HOG 刻意不做方向對齊——這對「不變性是不是愈多愈好」有什麼啟示?

    提示與參考解答：共同骨幹都是「梯度方向直方圖」。三個差異：(1) 取樣——SIFT 稀疏（只在關鍵點，數量每張不固定）、HOG 稠密（固定網格全覆蓋，每個視窗維度固定）;(2) 方向——SIFT 先量主方向再對齊以取得旋轉不變，HOG 刻意不對齊;(3) 用途——SIFT 要解對應問題所以必須配對（比值測試 + RANSAC），HOG 要解辨識問題所以直接把固定長度的向量餵給分類器。「維度固定」這件事很關鍵：分類器要求每個樣本的維度一致，而關鍵點的數量每張影像都不同，沒辦法直接餵。HOG 不做方向對齊，是因為它要辨識的是「站著的行人」，把每個區塊都轉正等於丟掉了「這個人是直立的」這個最重要的線索。啟示：不變性不是愈多愈好。每多一分不變性，就少一分鑑別力——該對什麼不變，是由任務決定的，不是由演算法的優劣決定的。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 11(特徵擷取)。

- Lowe, D. G., Distinctive Image Features from Scale-Invariant Keypoints, IJCV, 60(2), 2004.(SIFT;US Patent 6,711,293，本章叫獸開講之史料來源)
- Harris, C., & Stephens, M., A Combined Corner and Edge Detector, Alvey Vision Conference, 1988.
- Shi, J., & Tomasi, C., Good Features to Track, CVPR, 1994.
- Rublee, E., et al., ORB: An Efficient Alternative to SIFT or SURF, ICCV, 2011.
- Fischler, M. A., & Bolles, R. C., Random Sample Consensus (RANSAC), Comm. ACM, 1981.
- Dalal, N., & Triggs, B., Histograms of Oriented Gradients for Human Detection, CVPR, 2005.
- Ojala, T., Pietikäinen, M., & Mäenpää, T., Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns, IEEE TPAMI, 24(7), 2002.
- Hu, M.-K., Visual Pattern Recognition by Moment Invariants, IRE Transactions on Information Theory, 8(2), 1962.
- 補充教材與課程資訊：@prof(https://page.line.me/prof)